Epiphany

Operation Catalog

A companion to the Core Specification

Version 0.21.0 — Expression and text

Normative for the operation kinds it defines

Contents

1 About This Companion **3**
- 1.1 Relationship to the Core Specification . . . 3
- 1.2 Conformance Profiles . . . 4

2 The Catalog Framework **12**
- 2.1 Value-Typed Payloads . . . 12
- 2.2 Per-Primitive Schema Template . . . 12
- 2.3 Reduction-Discipline Coverage . . . 13
- 2.4 The Two Reduction Modes . . . 13

3 K0 — Representative Primitives **15**
- 3.1 InsertEvent . . . 15
- 3.2 DeleteEvent . . . 16
- 3.3 RespellPitch . . . 17
- 3.4 ModifyEvent . . . 17
- 3.5 Identified-Pitch Operations . . . 19
- 3.6 Transpose (frozen; replay only) . . . 20
- 3.7 TransposeInterval . . . 21
- 3.8 CreateCrossCutting . . . 24
- 3.9 DeleteCrossCutting . . . 26
- 3.10 ModifyCrossCutting . . . 26
- 3.11 ChangeRegionTimeModel . . . 27
- 3.12 Structural Containers . . . 28
- 3.13 CreateStaff . . . 29
- 3.14 CreateInstrument . . . 30
- 3.15 SetCanvasLayoutDefaults . . . 31
- 3.16 SetSpellingPrecedence . . . 31
- 3.17 SetTuningContext . . . 32
- 3.18 CreateStaffGroup . . . 33
- 3.19 CreatePartDefinition . . . 34
- 3.20 CreateAnalysisLayer . . . 35
- 3.21 CreateView . . . 35
- 3.22 CreateMeasure . . . 36
- 3.23 CreateTuplet . . . 37
- 3.24 SetClef and SetKeySignature . . . 39
- 3.25 SetVoiceHome . . . 39
- 3.26 Repeat Structures . . . 40

3.27 SetUserSystemBreak . . . 41
3.28 Score Settings . . . 41
3.29 Meter and Tempo Overwrites . . . 42
3.30 SetStaffLayout . . . 43
3.31 DeclareTransaction . . . 44
3.32 ResolveConflict (meta-operation) . . . 44
3.33 ResolveEquivocation (meta-operation) . . . 45
3.34 UndoTransaction (meta-operation) . . . 46

4 v0 → v1 Payload Migration 48

5 K1 — Framework Slots (Phase 3) 49

Chapter 1

About This Companion

The *Operation Catalog* is a companion to the Epiphany Core Specification. It fulfils the open question the core specification raises in its *Operation Catalog Conformance* section (Chapter 6, Semantic Operations and Concurrent Reduction, the `sec:semops:catalog` open question), which states that the catalog "is normative once published; this specification is non-final until the catalog is delivered." This release delivers the catalog *framework*, the **Ko representative primitive set**, and the **M2 broad-Ko groups** (the event/pitch leaf-field, cross-cutting CRUD, structural-container, and score-settings operations the Phase 2 slice exercises) — the operation kinds the Phase 2 visible slice and binary format actually exercise, each fully specified in Chapter 3. The remaining items of the full 60–80-primitive catalog are drafted as framework slots (Chapter 5) and completed in Phase 3.

1.1 Relationship to the Core Specification

This companion does not restate the operation framework; it *references* it. The framework — operation identity and stamps, the hybrid-logical-clock monotonicity rule, the dotted-version-vector causal context, the order-independent operation-slot model and equivocation, the canonical reduction order, the four-phase lifecycle, conflict records and the conflict registry, re-anchoring, transactions, forward undo, and the LWW discipline — is the core specification's Chapter 6. This companion defines, for each operation kind, the *payload schema* and how that kind *instantiates* the framework's reduction, conflict, undo, and re-anchoring rules.

It also consumes, rather than re-deriving, the **ratified byte-convention baseline** (core specification Chapter 8, Binary Format Companion, requirement `req:format:codec-conventions`, and the Canonical Byte-Layout Reference appendix): little-endian integers, a single discriminant byte per tagged union, `u32` length prefixes on every variable-width leaf, and raw UTF-8 free text. Operation-payload encodings are expressed in terms of that baseline; the literal wire layout of each payload is the Binary Format companion's (Agent J's) to pin, in coordination with this catalog.

Rationale

The catalog is versioned *independently* of the core specification (independent semver). Operation kinds are added over time; the catalog should evolve — adding primitives, refining conflict cases — without forcing a core specification revision. The core specification changes only when the *framework* changes.

1.2 Conformance Profiles

A **Phase-2 profile** implementation **MUST** implement every primitive in Chapter 3 — the representative set and the M2 broad-K0 groups — with the schema, reduction rule, conflict cases, undo semantics, and re-anchoring behaviour defined there. The former create-score/create-canvas framework slots of Chapter 5 are *retired* (Pass 12): they are permanently outside the operation set, so no operation kind exists to reject.

Version 0.6.0 (Pass-12 G-ratification). No payload byte changes. Ratified: ResolveConflict beyond the concurrent case (P12-K4), ResolveEquivocation edge semantics (P12-K6) and the profile-policy deferral (P12-K5), the RespellPitch migration fallback as long-term (P12-K1), undo strand-block conflict-kind reuse (P12-K10) and the idempotence asymmetry (P12-K11), the Transpose prototype pin (P12-K2 deferral), the cross-region slur AND advisory (P12-K12), and the create-score/canvas retirement (P12-K8). Appended vocabulary (minor, append-only): `SystemDerivedContentImmutable` (12, P12-K3) and `RecreateContentMismatch` (13, P12-K9) in `PreconditionFailureReason`.

Version 0.8.0 (Push 4a, transpose algebra). Adds one operation kind, `TransposeInterval` (Section 3.7), carrying the existing `TranspositionInterval` of core Chapter 2. It transposes scale position, refuses atomically rather than saturating, takes a genuine *set* of targets, and propagates the resulting spelling. `Transpose` (Section 3.6) is **frozen**: its saturation, its nominal-blindness, and its duplicate-sensitivity are pinned as normative replay semantics and no longer authored. This supersedes the 0.6.0 pin of the Transpose prototype (P12-K2), which anticipated a payload schema-major landing with the tuning catalog; neither proved necessary. Appended vocabulary (minor, append-only): `AcousticRealizationPinned` (14) and `TranspositionOutOfRange` (15) in `PreconditionFailureReason`; `PitchSpaceMismatch` (6) is un-reserved and now produced for a non-`Cmn` position. That case reads the position discriminant and does not require a pitch-space registry.

Version 0.9.0 (P13-S2, space-relative CMN). Broadens the `PitchSpaceMismatch` (6) case for `TransposeInterval`: a `Cmn` target whose enclosing chromatic structure or nominal mapping cannot be established now refuses under core requirement `req:pitch:space-capability-refusal`, rather than receiving guessed 12-chromatic arithmetic. This capability check is distinct from the 0.8.0 non-`Cmn` discriminant check and is replaced by structural registry resolution in Push 4b. No payload bytes change and no `PreconditionFailureReason` is appended; assignments 10 through 15 remain exactly as ratified.

Version 0.10.0, first half (genesis tranche G1, retroactively recorded). `CreateInstrument` (Section 3.14) landed at commit `3b09595` without a version bump or a changelog paragraph, against this document's own convention above — an omission genesis tranche G2a repairs here rather than silently. Adds one operation kind, `CreateInstrument`, mint-only (set-union creation, no delete in this catalogue revision) on the score root, carrying the full `Instrument` value. It is the single missing link between an empty score (`Score::empty`) and a note: `CreateStaff` already demands a live `Instrument`, and nothing else could mint one. Schema major 2 unconditionally (`Instrument`'s major-2 appends are mandatory fields, not `Option`-hidden).

Version 0.10.0, second half (genesis tranche G2a). Adds two operation kinds, `SetCanvasLayoutDefaults` (Section 3.15) and `SetSpellingPrecedence` (Section 3.16), on exactly the `SetMetadata` advisory-

LWW pattern (Section 3.28): the canvas's default layout advisories and the score's spelling-precedence configuration join the operation-authored surfaces. Both are schema major 0 unconditionally — neither carried type's canonical encoding, taken as a standalone payload, has ever been versioned. No accept-set change: the third genesis-ladder setter, `SetTuningContext`, is schema major 3 and is deliberately deferred to its own rung (G2b) rather than anticipated here.

The *Value restoration* passage (Section 3.34) and the *Retired slots* closing note both gain a repair alongside this addition: the former's LWW-family enumeration now names both new setters, and the latter's "outside the operation set" clause is narrowed to the canvas object itself, per `spec/RULING_GENESIS_PERSISTENCE.md` — the score's *contents* are no longer outside the operation set; the canvas is still never op-minted.

Version 0.11.0 (genesis tranche G2b, `spec/CONTRACT_GENESIS_G2B_TUNING.md`). Adds one operation kind, `SetTuningContext` (Section 3.17), the third and final genesis-ladder setter deferred at 0.10.0. Carries `TuningContextSettings`, the authored subset of `ScoreTuningContext` — exactly its five wire-bearing fields, byte- identical to the existing schema-major-3 wire walk (Binary Format companion SCHEMA MAJOR 3) — rather than the full graph type, so that `accidental_extensions` (staged off the wire) cannot diverge between a live-authored session and the same document reloaded from bytes.

The sole accept-set-raising surface among the nine genesis settings/creates. `SetTuningContext` is schema major 3 *unconditionally* (`ScoreTuningContext`'s `smufl/ overrides` appends are mandatory, so no lower-major layout for this payload exists at any value), which is why it alone — not `CreateInstrument` at major 2, and not either G2a setter at major 0 — drags the `OperationEnvelopeBlock` accept-set from `[0, 2]` to `[0, 3]` (Binary Format companion THE CHUNK-LEVEL GATE). This is the reason the genesis ladder split `SetTuningContext` into its own rung rather than landing it alongside `SetCanvasLayoutDefaults` / `SetSpellingPrecedence`: a one-way accept-set door is not something to bury inside two unrelated major-0 setters.

Undo restores the **seeded base** settings — default or non-default alike — exactly as `SetMetadata` does; the never-authored / authored-to-default distinction stays unobservable (Section 3.17). This closes P13-S13: the canonical operation log is now the sole canonical carrier of the tuning context (the canonical base embeds no `Score` field value for any setting, `metadata` included), on the same reasoning that already applied to `SetMetadata`.

Version 0.12.0 (genesis tranche G3a, `spec/CONTRACT_GENESIS_G3A_ENTITIES.md`). Adds four operation kinds — `CreateStaffGroup` (Section 3.18), `CreatePartDefinition` (Section 3.19), `CreateAnalysisLayer` (Section 3.20), `CreateView` (Section 3.21) — completing the root-level `Score` entity mints: every remaining vector a Chapter 5 top-level structure declares (`staff_groups`, `parts`, `analysis_layers`, `views`) is now operation-authored. All four are mint-only (set-union creation, no delete in this catalogue revision), schema major 0 unconditionally, epoch 11.

The defect this closes, and what it does not close. Until this rung, all four object kinds became live *only* through canonical-base ingest; no operation minted any of them. Consequently `CreateStaff`'s `group` precondition (Section 3.13) was **unsatisfiable** under from-empty reduction — a document built only from operations could never author a grouped staff. G3a closes exactly this satisfiability half. It does **not** make a bidirectionally consistent staff group authorable: with mints only, and no modify or delete operation for either side, no authoring

order can bring `Staff.group` and `StaffGroup.members` into agreement once they disagree.

Ruled (disposition A, P13-S16; supersedes disposition B): `Staff.group` is the sole authority for group membership, and `StaffGroup.members` is a projection maintained from it by reduction. `CreateStaffGroup` **refuses** a non-empty carried `members` (`ContainerNotEmpty`) — the operation authors the group, not its membership — and `CreateStaff` carrying `group: Some(g)` appends the new staff to `g`'s `members` in the graph. The two **MUST** agree in both directions, and graph invariant 21 (core specification Graph Invariants) flags any disagreement between live objects. Every consumer **MUST** read group membership from `Staff.group`; core specification Chapter 5's `Staff`/`StaffGroup` declarations (Top-Level Score Structure) state this normatively, and Section 3.13 above states the corresponding authoring-order consequence for `CreateStaff`.

What disposition B ruled, and why it no longer holds. Genesis tranche G3a ratified disposition B: `members` was stored exactly as carried and **neither maintained nor trusted**, and **both** stale forms were permitted outcomes rather than defects — a **missing** member (`Staff.group` naming a group whose `members` omits that staff) and a **spurious** member (`StaffGroup.members` naming a staff whose own `Staff.group` is absent or names a different group). That ruling was sound while the tranche minted only: with no modify or delete operation for either side, no authoring order could bring the two into agreement once they disagreed. Disposition A was recorded there as the later maintenance/enforcement fix, sequenced after G3b; **P13-S16 is that fix**, and neither stale form is authorable any more.

Referential preconditions, graph-aware only, mirroring `CreateStaff` exactly. `CreatePartDefinition.staves` preconditions every named id resolves to a live `Staff`; `CreateView.active_layers` preconditions every named id resolves to a live analysis layer; `CreateAnalysisLayer` carries no outbound reference and needs no referential precondition at all, the `CreateInstrument` shape rather than the `CreateStaff` shape. Base-free reduction enforces none of these, exactly as for `CreateStaff`: there is no universe to check against. No new `PreconditionFailureReason` — every referential refusal reuses `TargetMissing` (discriminant 0).

`CreateStaffGroup` is deliberately not in that list (P13-S16). Its `members` check is an **empty-container** precondition on the carried value, not a referential one, and the distinction is normative: a graph-aware precondition asks about the *universe* ("is this id live?") and therefore cannot run base-free, whereas an empty-container precondition asks only about the *carried value* ("is `members` empty?") and therefore **MUST** be enforced everywhere, base-free included. It refuses with `ContainerNotEmpty`, and the sentence above about base-free reduction enforcing none of these does **not** extend to it.

No accept-set change: all four carried types have exactly one wire layout (core specification's Binary Format Companion confirms no versioned walk exists for any of the four), so none gains a `schema_major()` arm beyond the existing catch-all.

Version 0.13.0 (genesis tranche G3b, `spec/CONTRACT_GENESIS_G3B_MEASURE.md`), the final rung of the genesis operation tranche (`spec/PLAN_GENESIS_OPS.md`). Adds one operation kind, `CreateMeasure` (Section 3.22): `StaffInstance.measures` joins the operation-authored surfaces, on the `CreateStaffInstance` parent-carrying mint pattern rather than the bare-value shape the four G3a families above use — `Measure` is a nested container child, not a `Score`-root vector, and carries no back-pointer to its parent. Schema major 0 unconditionally, epoch 12.

Also adds graph invariant 20 (core specification Graph Invariants) — measure/meter agree-

ment and boundary consistency — and three `PreconditionFailureReason` appends, all epoch 12: `MeasureMeterMismatch` (16), `MeasureOutOfOrder` (17), and `MeasureOrderUnverifiable` (18). `CreateMeasure` itself refuses whenever the comparison or delta invariant 20 needs is not computable (`MeasureOrderUnverifiable`); invariant 20 instead *abstains* — emits no violation — over the same incomputable cases, because base-ingested data may predate the rule. This is a deliberate split, not an inconsistency: fail closed at the operation, abstain at the invariant.

Two existing operations now preserve invariant 20. `SetMetricGrid` (Section 3.29) and `SetTimeSignature` (Section 3.29) both gain prospective preconditions, covering agreement and boundary consistency, on their forward paths and on both undo restoration policies (`StrictInverse/Cascade` conflict on an unsafe restoration set; `BestEffort` applies the maximal safe subset under a documented canonical-order greedy). Every check runs before any mint, so a refused write leaves no residue in `objects`, any carried-value map, or the graph.

No accept-set change: `Measure` has never been versioned as a standalone value — its only non-scalar field, `TimeAnchor`, has a hand-written codec with no version branching of its own — so `OperationEnvelopeBlock` stays at `3` where genesis tranche G2b left it. This closes the genesis ladder (`spec/PLAN_GENESIS_OPS.md` §4): G1 → G2a → G-minor → G2b → G3a → G3b.

Version 0.14.0 (P13-S19, correction and observation, `spec/CONTRACT_P13S19_PARTIAL.md`). No payload byte changes, no behaviour change: `create_measure` and graph invariant 20 are byte-identical to 0.13.0. Corrects this section's ordering and boundary-distance description above, which read as though a pickup/anacrusis first measure — and everything downstream of it — were simply exempt. A first measure skips only the *predecessor-dependent* clauses (ordering and boundary distance) for lack of a predecessor, and the agreement clause only when it declares `None` or a signature that agrees with the governing grid; a disagreeing declared signature is refused (`MeasureMeterMismatch`) even on a first measure, since agreement is not predecessor-dependent. Independently of what the pickup itself declares, its **successor** is measured against the governing signature's full `measure_duration()` regardless, and is refused (`MeasureMeterMismatch`) and flagged by invariant 20 whenever the pickup's actual length is shorter than a full bar — which, for an authored pickup, it always is. The same gap applies to any partial measure at any position, not only a first one: a mid-score partial measure is itself unexamined at creation (nothing checks its own length), while *its* successor is refused the identical way. Closing this needs a per-measure duration concept this rung does not introduce (`spec/PASS13_CANDIDATES.md`, P13-S19, still open).

Version 0.15.0 (P13-S16, the maintained projection, `spec/CONTRACT_P13S16_PROJECTION.md`). No payload byte changes — `CreateStaffGroupOp` and `CreateStaffOp` encode exactly as in 0.12.0 — but a **behaviour change on two operations**, replacing genesis tranche G3a's ratified **disposition B** with **disposition A**. `StaffGroup.members` stops being a non-authoritative field stored as carried and becomes a projection *maintained* from `Staff.group`:

- `CreateStaffGroup` (Section 3.18) now **refuses** a non-empty carried `members` with `ContainerNotEmpty` (discriminant 10, no new `PreconditionFailureReason`). This **replaces** the graph-aware precondition that every id in `members` resolve to a live `Staff` (`TargetMissing`). The new check is an *empty-container* precondition on the carried value, so unlike the check it replaces it is **not graph-gated and is enforced base-free** — the one place where the sibling mints' "referential preconditions are graph-aware only" rule does not extend.

- `CreateStaff` (Section 3.13) carrying `group: Some(g)` now appends the minted staff to `g`'s `members` in the graph, idempotently per staff id. Undo of a staff strips the id back out.

Both formerly **permitted** stale forms — the missing member and the spurious member — are therefore **unauthorable**, and the core specification gains graph invariant 21 (`StaffGroupMembershipAgreement`) requiring the two to agree in both directions between live objects, abstaining on dangling membership because an undeclared member is invariant 10's concern.

This breaks documents authored under 0.12.0–0.14.0 only in what they may newly assert, not in what they may read: an existing blob carrying a disagreeing pair still decodes, and no stored bytes are reinterpreted.

A canonical base materialized before this rung MUST be rebuilt rather than reused, and the two operations oblige that for *different* reasons, either of which alone would suffice:

- `CreateStaffGroup` changes its **reduction verdict** — a non-empty carried `members` now reduces to a no-op where it previously applied, so the recorded effect differs.
- `CreateStaff` changes **canonical reduced state** — its verdict is unchanged (it still applies), but the graph the reduction produces now carries the maintained `members`.

Accordingly `epiphany_ops::CURRENT_REDUCTION_ALGORITHM_VERSION` moves `0` → `1`, and P13-S27's authority check refuses a base declaring the older version. Core specification Canonical Document Identity requires the bump for either class of change.

Version 0.16.0 (X3.1, tuplets). Adds one operation kind, `CreateTuplet` (Section 3.23): the score's tuplets join the operation-authored surfaces, on the root-level mints' bare-value shape. Like repeats, tuplets live in the cross-cutting registry but are not `CrossCuttingValue` kinds on the wire. Schema major 0 unconditionally, epoch 13; wire discriminant 40. No new `PreconditionFailureReaso` a tuplet its members do not fill refuses with `EventDurationInvalid`.

Two existing operations now preserve tuplet consistency. `DeleteEvent` (Section 3.2) reads a member's tuplets from the graph-independent referent index rather than the graph, so a member's delete without its compensation is refused base-free as well as graph-aware (before this revision only graph-aware reduction could see a tuplet, and no operation could make one); base-free reduction also refuses, as graph-aware reduction does, a `RewriteTuplets` compensation, a `CascadeDeleteTuplets` that does not name exactly the live tuplets holding the event, and a `ReplaceWithRest` whose rest's duration differs from the event's, read from the graph-independent occupancy index; and its `ReplaceWithRest` compensation puts the rest in the deleted member's place in the referent index. `ModifyEvent` refuses a change to a tuplet member's duration (`EventDurationInvalid`). A member tombstoned by an undo cascade-deletes its tuplet.

A pickup's successor now applies (X3.5, P13-S19). `CreateMeasure`'s boundary-distance precondition (Section 3.22) measured a first measure's successor against a full bar, so after a pickup every later measure was refused `MeasureMeterMismatch`. While the predecessor is the instance's only live measure, the successor's delta may now also be any positive duration less than the governing signature's `measure_duration()`: a first measure may be a pickup. A short measure after the first is still not modelled, and its successor still refuses.

A canonical base materialized before this rung MUST be rebuilt rather than reused. On histories of the operation kinds that existed before it, eight verdicts change and two operations

change the state they produce. The verdicts: a base-free `DeleteEvent` declaring `RewriteTuplets`, or `CascadeDeleteTuplets` not naming exactly the live tuplets that hold the event, is refused `TupletCompensationInvalid`, where it applied with a compensation recorded against a tuplet no operation had minted (and, for the cascade, that identifier tombstoned); a base-free `DeleteEvent` whose `ReplaceWithRest` rest differs in duration from the event is refused `TupletCompensationInv` as graph-aware reduction refuses it, where it applied (with tuplets made base-free, a concurrent trim makes such a rest on a valid history, which the two modes would otherwise reduce differently); a `ModifyEvent` changing a tuplet member's duration over a base holding the tuplet is refused `EventDurationInvalid`, where it applied and broke the tuplet's duration consistency; a pickup's successor applies (above); a base-free `ChangeRegionTimeModel` conflicts `TimeModelMigrationFailure` when its region holds metric events its target does not admit or a `Reassign` leaves unmapped, where it applied; after an applied `Reassign`, base-free reduction reads the remapped placements (in an insert's overlap check, a move's placement verdict, a replacement rest's placement and the re-anchoring order), where it read the old ones; a base-free `InsertEvent` into a region that is not metric is refused `WrongRegionTimeModel`, where it applied or was refused for another reason (an overlap, or a carried pitch identifier already in canonical state); and graph-aware, a migration judges an event the occupancy index holds by its indexed placement, which differs from the graph only for an event inserted at a wall-clock position into a metric region (already a breach of the coordinate discipline), now admitted by a metric target where the migration conflicted. The last four are older than this revision: the two reduction modes disagreed through `ChangeRegionTimeModel`, and now agree (Section 3.11). The states, over a base that already holds a tuplet: a `DeleteEvent` whose `ReplaceWithRest` compensation replaces a member: the index used to keep naming the deleted member, so the re-anchoring pass recorded an `AttachmentTombstoned` repair against the tuplet the graph had just given the rest; it records none now. And a member tombstoned with no compensation to declare, as a cue event cascaded out from under its tuplet is, or as a replacing rest is when an undo removes the transaction that minted it: the tuplet used to stay, naming a dead member; it now cascade-deletes, in the effect and the graph. Accordingly `epiphany_ops::CURRENT_REDUCTION_ALGORITHM_VERSION` moves $1 \to 2$.

Version 0.17.0 (X3.6, clef and key changes). Adds two operation kinds, `SetClef` and `SetKeySignature` (Section 3.24): a staff instance's clef and key sequences, written until now only by `CreateStaffInstance`, take a change at a time. Schema major 0 unconditionally, epoch 14; wire discriminants 41 and 42. No new `PreconditionFailureReason`. The reduction algorithm version does not move: no operation reducible before changes its verdict or the state it produces, since only the new kinds write the new registers.

Version 0.18.0 (X3c, a tuplet's display). `CreateTuplet` (Section 3.23) carries the tuplet's `display`, which schema major 4 appends to `Tuplet`, so it is schema major 4 unconditionally where it was major 0; a `CreateTuplet` written before major 4 is refused by name. No discriminant, precondition or reduction rule changes. The reduction algorithm version does not move: the display enters no verdict and no canonical reduced state, so every history reduces as before, and only the graph-aware score holds the display.

Version 0.19.0 (X4a.2, the two reduction modes agree). Base-free reduction and graph-aware reduction onto an empty base are held to each other, in every effect, the objects and the canonical bytes, over concurrent histories that editors write, each operation naming only what its author has seen; each verdict on which they parted is settled here, in the mode that dis-

agreed. `InsertEvent`'s promotion pre-pass takes every concurrent insert into a voice, whatever its preconditions (Section 3.1): graph-aware reduction took only inserts whose voice its graph held before anything applied, so over an empty base it promoted none, refusing as an overlap the insert base-free reduction promoted. `TransposeInterval` moves an authored spelling on a quarter-tone (Section 3.7): it refused any authored spelling on a `cmn-24` pitch, which every quarter-tone the importer reads carries, graph-aware, where base-free reduction, holding no spelling, applied the transpose. And base-free reduction reads a referent its set mints as the set leaves it (Section 2.4, Requirement 2.2): each referential precondition, checked graph-aware only before, is checked in both modes, an object no envelope mints still taken as live. Base-free reduction also holds each pitch it minted at its current value, so a `TransposeInterval` writes its targets' values in both modes: its concurrency with a `ModifyIdentifiedPitch` of the same pitch, and an undo of it, reduce alike (Section 3.7), where base-free reduction recorded no write and so applied the edit and refused the undo. A transposition refuses a value no accidental stack writes, from the value in both modes, and drops an authored spelling it cannot move rather than refusing on it; a deleted pitch takes every spelling attachment scoped to it, a transposition's propagated one included. An undo reads the same write chains in both modes: base-free reduction seeds the score-level settings with an empty score's values, where it restored them to absence, and records every write to a pitch's spelling set (Requirement 3.5), where it recorded none. A `ChangeRegionTimeModel` whose `Reassign` would leave two events of one voice overlapping conflicts, naming both (Section 3.11), where it applied. A `CreateStaffInstance` for a staff its region already manifests is refused with `ContainerNotEmpty`, where two concurrent creates both applied. A `DeleteRegion` of a region another map's live tempo segment anchors to is refused with `ContainerNotEmpty`; a migration to a model admitting no musical offset conflicts on the region's measures as on its events, and applied drops the region's breaks; and a break written in musical time into such a region is refused with `WrongRegionTimeModel`, each where an anchor was left naming nothing or a time the region no longer has. An undo cannot remove a measure a later measure of its instance follows, which would leave that measure two bars from its predecessor, nor a region, staff instance or voice another transaction has filled, and an undone region or instance leaves the graph; an undo restoring a cross-cutting value whose endpoint was deleted since is superseded by the delete; and a time signature or metric grid set in a region admitting no musical offset is refused with `WrongRegionTimeModel`, a migration into such a model dropping the region's grids, as is an insert carrying a wall-clock position. A tempo segment's anchors are read as referents and held to their region's time model, and a segment anchored in musical time strands its region's migration out of it. A `Reassign` that reorders a voice leaves it in position order. A tie still broken when its transaction completes gives way, removed with a recorded repair (Requirement 3.6), where the graph kept it broken. A `ModifyEvent` follows observed-remove, removing with a recorded repair a pitch its author saw and left out and keeping one a concurrent author added (add wins), and an undo restoring an event's value brings back the pitches the undone modifications removed; neither revives one a delete or undo removed (Section 3.4), where each dropped the pitch from the graph and left it live, or brought a tombstoned one back. A clef or key change, written at a musical offset, is refused in a region out of musical time, whether set or carried by a new staff instance, and a live one strands its region's migration out of it, where each left a change anchored by a musical offset the region did not admit. An undo cannot remove a staff a live part definition or spanner names, and a spanner's staves are read as referents, where each left a reference to a staff the score did not declare. A live tuplet supersedes an undo's restoration of its mem-

ber's earlier duration, where the restoration broke the tuplet's sum. An undo cannot remove a region a tempo segment of another map anchors to, nor an instrument a staff instance's override names, where each left the reference naming nothing. A whole-event write revives no pitch a delete removed, and a measure in musical time is refused in a region out of it. A region's whole grid and its meter changes compare by the order the reduction applied them. A best-effort undo that keeps a mint for what names it keeps what that mint names, where it removed the instrument of a staff it kept. A pitch inserted into an event of any kind without a pitch list, or kept by a whole-event write on one, makes it a note. A region or staff instance an undo emptied is empty to a later delete. A best-effort undo keeps a time signature a meter change it leaves in place still names, and an event an undo removes takes every pitch it holds, as a delete does. A promoted voice holds its instance against a delete, as any voice does. An undo that is a member of the transaction it names is refused, and a migration out of musical time supersedes an undo's restoration of a value written in musical time into its region. An undo's restoration naming an object tombstoned since is superseded by its tombstoning, as one naming a deleted endpoint was, a cross-cutting value's staves and measure or region anchors included. A whole-event modification mints a pitch it carries that nothing minted, where the pitch reached the graph alone, and a trajectory's own pitches are read and written as a note's, where a tie on one gave way graph-aware alone. A migration judges each of its region's events against the target's coordinate discipline as invariant 4 does, so an aleatoric target anchored in wall-clock time conflicts on an event in musical time, where it applied and left the event in coordinates the region no longer admitted. Two requirement labels are added; no payload, discriminant or `PreconditionFailureReason` changes. Accordingly `epiphany_ops::CURRENT_REDUCTION_ALGORITHM_VERSION` moves `2` → `3`.

Version 0.20.0 (X4b.0, a pitch belongs to one event). A `ModifyEvent` whose value carries a pitch another event holds live is refused with `RecreateContentMismatch` in both reduction modes (Section 3.4), where it applied and left the pitch in two events (`PitchIdUnique`, Chapter 5 invariant 12). No payload, discriminant or `PreconditionFailureReason` changes. Accordingly `epiphany_ops::CURRENT_REDUCTION_ALGORITHM_VERSION` moves `3` → `4`.

Version 0.21.0 (X4b.2, expression and text). Schema major 5 (Binary Format companion Schema Major 5) reaches the operations. An event's marks, ornaments and grace ride `InsertEvent` and `ModifyEvent`, which stamp major 5 where their value holds one; a grace note's timing is ruled in both reduction modes (Section 3.1). `CrossCuttingValue` appends `Marker` (discriminant 4) and `Lyric` (5), stamped major 5, with one syllable per event and verse (Section 3.8). `PreconditionFailureReason` appends `SlotOccupied` (discriminant 19, epoch 15) and, after its first review, `HomeIsOwnStaff` (20, epoch 15). `SetVoiceHome` (kind and tag 43, epoch 15) writes a voice's home staff (Section 3.25). The new verdicts join reduction version `4`, which X4b.0 opened and which does not move again before X4b merges. X4b's first fix round, after its first review, adds to it under this version: an undo restores no lyric syllable into a slot another holds (Section 3.10), and a marker its event's delete orphans holds its region (Section 3.8), and a home naming its voice's own staff is refused, `PreconditionFailureReason` appending `HomeIsOwnStaff` (discriminant 20, epoch 15; Section 3.25). Its second fix round, after its second review, adds one more: an undo reads a marker on an event it removes as the repair will leave it, so a marker the undo would orphan holds its region (Sections 3.8 and 3.34).

Chapter 2

The Catalog Framework

2.1 Value-Typed Payloads

Every operation payload in this catalog is **value-typed**: it carries the real graph values its effect introduces, not identifiers that point at values in some ambient graph. An `InsertEvent` carries the whole `Event`; a `RespellPitch` carries the whole `PitchSpelling`; a `CreateCrossCutting` carries the whole tie, slur, beam, or spanner.

Rationale

The vo prototype carried *identifier-only projections* — an `InsertEvent` held an `EventId` plus reduction-relevant scalars; a `RespellPitch` held a content-hash *fingerprint* of the new spelling. That was sufficient to make an envelope hashable and to drive reduction, but it is **not durable**: an operation replayed in a fresh context — a backup restore, a cross-tool round-trip — needs the full value, which an identifier-only payload cannot supply. Value-typed payloads are what make an Epiphany document portable.

Requirement 2.1

A value-typed payload field **MUST** be encoded by emitting the field's value under the core specification's canonical value encoding (`req:format:codec-conventions`), framed by a `u32` little-endian length prefix. Decoding **MUST** be the exact inverse and **MUST** reject trailing bytes within the framed region. The encoding introduces no byte layout beyond the ratified baseline: a value's bytes here are byte-for-byte the bytes the whole document codec emits for that value.

2.2 Per-Primitive Schema Template

Each catalog primitive is specified under a fixed six-part template. A new primitive is a schema-fill against this template, not a fresh design:

Payload schema The value-typed fields the operation carries, with their core-specification types.

Canonical encoding The field order and framing, consuming Requirement 2.1. (The literal byte layout is the Binary Format companion's; this catalog fixes the *field set and order*.)

Reduction rule How the operation mutates canonical state when it is reached in canonical reduction order — the preconditions it checks, the objects it mints or tombstones, and the bookkeeping it records.

Conflict cases The conflict records the operation can produce, by `ConflictKind`, and which participant materialises.

Undo semantics The compensating effect of undoing a transaction that contains the operation, under each `UndoPolicy` (`StrictInverse` / `BestEffort` / `Cascade`).

Re-anchoring The behaviour when an object the operation references is tombstoned before or concurrently with it.

2.3 Reduction-Discipline Coverage

The Ko representative set is chosen so that, between them, the primitives exercise *every* reduction discipline the framework defines: position-keyed insert with system-voice promotion; delete-wins with tombstones, tuplet compensation, and cross-cutting re-anchoring; field-overwrite with last-writer-wins and structural-field-collision conflicts; set-union creation; structural time-model migration; an LWW advisory; atomic transactions with descriptor precedence; and the two meta-operations (conflict resolution and forward undo). A primitive added later that reuses one of these disciplines inherits its reduction, conflict, undo, and re-anchoring treatment.

2.4 The Two Reduction Modes

An operation set is reduced either *graph-aware*, onto a canonical base whose objects are all known, or *base-free*, with no base and so no universe of objects beyond what the set itself shows. A history whose every operation names only what its author had seen, reduced from an empty base, is *valid*; on a valid history the two modes **MUST** agree in every effect, in object existence and in the canonical bytes (since version 0.19.0).

Requirement 2.2

Base-free reduction **MUST** read a referent an operation names as follows: one that no envelope of the set mints may come from a base, and is taken as live; one that an envelope of the set mints is live only if an operation of the reduction made it live and none has since tombstoned it, a losing candidate's mints and a refused operation's included among those minted; and a system-promoted voice is live only if a promotion of the reduction made it. Each referential precondition this catalog states (that a named staff, instrument, group, analysis layer, time signature, voice or region is live) is checked in both modes on that reading.

Rationale

Before version 0.19.0 these preconditions were checked graph-aware only, so an operation naming an object its history minted and lost (deleted concurrently, or minted by a transaction that failed, or by an equivocation's losing candidate) applied base-free and

> was refused graph-aware, on a valid history. Base-free reduction still has no universe: an object no envelope mints is never refused for being unknown.

Chapter 3

Ko — Representative Primitives

This chapter is normative under the Phase-2 profile. Each primitive's reduction, conflict, undo, and re-anchoring behaviour is the behaviour the core specification's Chapter 6 defines for its discipline; the description here states how the primitive instantiates it. The reference implementation is `epiphany-ops` (the `payload`, `reduce`, and `migrate` modules).

3.1 InsertEvent

Payload schema. `InsertEventOp { staff_instance: StaffInstanceId, event: Event }`. The voice, region-local position, duration, and pitch identities are read from the `Event` value; the `staff_instance` is retained alongside it so the system-voice promotion derivation is total without a containment walk.

Canonical encoding. `staff_instance`, then the length-framed canonical bytes of `event`.

Reduction rule. A position-keyed insert. Preconditions: the event's duration is positive, or zero for a grace note, and zero only for one (see *Grace notes* below); its position is musical, a wall-clock one refused with `WrongRegionTimeModel` in both modes (since version 0.19.0, before which it was admitted into a metric region at the region's origin); the event id is neither live nor tombstoned; the region of the event's staff instance is metric, read in graph-aware reduction from the graph and base-free from the reducer's index of each region's coordinate discipline, which `CreateRegion` sets and an applied `ChangeRegionTimeModel` moves, so the two modes agree after a migration; the target voice exists in that region (graph-aware, from the graph; base-free, as Requirement 2.2 reads it) and the event's pitch ids are fresh. The event and its pitches are minted live; the voice is created on first use. Concurrent inserts whose half-open duration intervals overlap in the same voice are resolved by an order-independent promotion pre-pass, which takes every such insert whatever its preconditions, in both modes (since version 0.19.0): the lower-`OperationId` insert is retained in the original voice, and each overlapping loser is promoted to the deterministic system voice `derive_promoted_voice_id(staff_instance, voice, winner, loser)` and tagged `VoicePromoted`. A promoted voice is a voice of the staff instance the insert names, in the reducer's index as in the graph, so the instance's delete is refused with `ContainerNotEmpty` while it holds an event (since version 0.19.0, before which the index left it out and the instance's delete applied over it).

Grace notes (since version 0.21.0). A grace note is an event whose `grace` is present (core specification, schema major 5); it occupies no voice time. Its duration is zero and only a grace note's is: an insert or a modification whose value breaks either is refused with `EventDurationInvalid`, in both modes. A zero-length span overlaps nothing, so graces at one position from any number of authors all stand, with no promotion; but a grace stands at an event's onset, never strictly inside another event's span in its voice, and an insert or a move that would put a grace there, or an event's span over a standing grace, is refused with `EventDurationInvalid`. A voice's events stand by position, a position's graces before its event of positive duration in their `order` and then by identifier. A grace's principal is positional, not a reference: a grace survives its note's delete, standing alone. A grace is no tuplet's member (Section 3.23), and a standard tie's ends are adjacent among the voice's events of positive duration, the graces before either taking no place.

Schema major. An insert whose event holds a mark, an ornament or a grace is stamped schema major 5; any other is stamped as before, its bytes unchanged.

Conflict cases. None at reduction time; promotion is a deterministic repair, not a conflict.

Undo semantics. Undoing the enclosing transaction tombstones the minted event, its pitches, and any promoted voice. `StrictInverse` conflicts if any was already tombstoned or modified; `BestEffort` tombstones the survivors; `Cascade` is `StrictInverse` over the same minted set (dependent-closure undo is a Phase-3 refinement). The event takes every pitch it then holds with it, as a `DeleteEvent` does, a pitch another operation added since among them (since version 0.19.0, before which such a pitch stayed live while the graph dropped it with its event).

Re-anchoring. Not applicable (the operation mints, it does not reference a pre-existing object that could be tombstoned).

3.2 DeleteEvent

Payload schema. `DeleteEventOp { event: EventId, tuplet_compensation: TupletCompensation }`, where `TupletCompensation` is `NotInTuplet`, `ReplaceWithRest { rest: Rest }` (value-typed replacement rest), `RewriteTuplets { tuplets }`, or `CascadeDeleteTuplets { tuplets }`.

Canonical encoding. `event`, then the tuplet-compensation discriminant and its payload (a length-framed `Rest` value for `ReplaceWithRest`).

Reduction rule. Delete-wins: the event and its contained pitches are tombstoned (retaining their identifiers). Tuplet compensation, when present, adds the replacement rest live or tombstones the cascaded tuplet group. Concurrent deletes of the same event are idempotent.

Conflict cases. None for the delete itself. Graph-aware reduction refuses an ill-formed tuplet compensation as a precondition failure (no-op). Base-free reduction refuses, from the referent index, a member's delete that declares no compensation, any `RewriteTuplets`, and a `CascadeDeleteTuplets` that does not name exactly the live tuplets holding the event; and, from the occupancy index, a `ReplaceWithRest` whose rest's duration differs from the event's, so the two modes reach one verdict on every compensation a conforming client declares. Whether the rest's identifier is fresh is a referential precondition, and like every referential precondition it is checked graph-aware only.

Undo semantics. An insert-shaped compensation re-introduces the tombstoned content; for the prototype's minted-object model this is the inverse of the tombstone set, with the policy treatment described under InsertEvent.

Re-anchoring. Tombstoning the event runs the framework's re-anchoring rule table over every cross-cutting structure that referenced it: a tie cascade-deletes; a comment or analytical annotation orphans (user content is never silently deleted); a beam truncates while ≥ 2 members survive and otherwise cascade-deletes; a slur or spanner re-anchors to the nearest surviving endpoint while ≥ 1 survives and otherwise cascade-deletes; a lyric cascade-deletes, and a marker re-anchors to the nearest live event of its staff instance or orphans to its region's start, in both modes (since version 0.21.0). A grace note before the event is not a referent: it stands alone.

3.3 RespellPitch

Payload schema. `RespellPitchOp { pitch: PitchId, spelling: PitchSpelling }` — the full spelling value (v1), not a fingerprint.

Canonical encoding. `pitch`, then the length-framed canonical bytes of `spelling`.

Reduction rule. A last-writer-wins field overwrite, keyed by pitch. Precondition: the pitch is live. The resolved spelling is the one carried by the operation latest in canonical order. Two respellings of one pitch that are causally ordered overwrite intentionally; two *concurrent* respellings with *equal* spelling reduce idempotently.

Conflict cases. Two concurrent respellings of one pitch with *differing* spellings produce a `StructuralFieldCollision` conflict recording the winner (later in canonical order), the loser, and the field `spelling`. The winner materialises and carries the `Conflicted` effect tag.

Undo semantics. Value-restoring (Section 3.34): undo restores the pre-operation spelling (or removes the spelling if the operation introduced the first one), under the active policy.

Re-anchoring. If the target pitch is tombstoned, the respelling is a no-op (`TargetTombstoned`).

Migration (ratified Pass 12, closing P12-K1). A v0 `RespellPitch` carried only a content-hash *fingerprint* of the spelling. The fingerprint cannot be inverted to a `PitchSpelling` without a side table, so the v0→v1 migration (Chapter 4) recovers the spelling from the score graph context — an explicit per-pitch spelling attachment whose canonical bytes hash to the fingerprint — and, when the context lacks it, declares the envelope unmigratable (the bundle opens read-only). This read-only fallback is the *long-term* disposition: no richer v0 corpus is or will be required (no production v0 corpus exists). It remains the one representative payload that is not self-contained under migration.

3.4 ModifyEvent

Payload schema. `ModifyEventOp { event: Event }` — the full replacement `Event` value (v1). The identity (and therefore the LWW key) is read from the value.

Canonical encoding. The length-framed canonical bytes of `event`.

Reduction rule. A last-writer-wins field overwrite keyed by event id. Precondition: the event is live. A pitch belongs to one event: a modification whose value carries a pitch another event holds live is refused as a clean precondition no-op (`RecreateContentMismatch`), as a create re-carrying a live identifier under a different parent is, read from the pitch index both reduction modes keep (since version 0.20.0, before which it applied and the pitch stood in two events, breaking Chapter 5 invariant 12). The resolved value is the one carried by the operation latest in canonical order; two causally ordered modifications overwrite intentionally, and two *concurrent* modifications with *equal* value reduce idempotently. Graph-aware reduction overwrites the event in place, and a modification that *moves* a metric event (a different region-local `Musical` position or duration) is *materialised*: the owning voice is re-sorted by ascending position (id-tiebroken — the same order an insert maintains), preserving `VoiceEventsSortedNonOverlap` (Chapter 5 invariant 3). To keep that invariant, a placement change carries a *placement precondition*, read from the reducer's canonical voice-occupancy index (graph-independent, so graph-free and graph-aware reduction agree): a move with a non-positive span, or one that would overlap another live event in the voice, is refused as a clean precondition no-op (`EventDurationInvalid`) rather than skipped silently. A grace note's value is held to the grace rules of Section 3.1 (since version 0.21.0): a modification giving a grace a positive duration, or an event of positive duration a grace, is refused with `EventDurationInvalid`, as is one that would leave a grace no pitch (Section 3.5); one that changes only a grace's `order` re-sorts its voice. A materialised move updates the occupancy index, so a later insert sees the freed or changed span. A placement change of a *non-metric* event is recorded in the bookkeeping but not applied to the graph (re-sorting a non-metric voice is a deferred refinement); a malformed (empty pitched) replacement is likewise recorded but not materialised; same-placement field edits apply in place, preserving voice membership. Partial trimming of a tuplet member remains a later refinement. The value sets the event's fields and its pitches' values, and its pitches follow *observed-remove* (since version 0.19.0, before which the value's pitches replaced the event's in the graph alone): a live pitch of the event the value does not carry is removed, as `DeleteIdentifiedPitch` removes one and with its attachments, if its author saw it, its insert being in the modification's causal past, and the effect records a `CascadeDeleted` repair for it; it is kept, at its current value and with its attachments, if its author never saw it (add wins). A pitch a base holds was inserted by no envelope of the set and the base precedes every operation, so it counts as seen (base-free reduction holds no base pitch). An event of any kind without a pitch list (a rest, an unpitched, indeterminate, graphic or cue event, a trajectory) written over a note its author saw so becomes that event, and over a note holding a pitch its author never saw becomes a note of that pitch, a trajectory's own pitches first (for every kind but a rest and an unpitched event since version 0.19.0, before which the graph dropped the kept pitch). A pitch the value carries that a delete or undo has tombstoned stays out of the event (delete wins), a trajectory carrying one becoming a note of its other pitches, as a delete of it would leave it. A pitch the value carries that no operation has minted is minted by the modification, as an insert mints its event's pitches: live, its value held in both modes, and under the modification's transaction, so an undo of it removes the pitch (since version 0.19.0, before which the pitch reached the graph alone and every operation naming it was refused). A pitch leaves an event only by `DeleteIdentifiedPitch`, `DeleteEvent`, a modification whose author saw it, or an undo of its insert.

System-derived content immutability (*ratified Pass 12, closing P12-K3*). A modification that would rewrite the *intrinsic content* of a pitch whose identifier lives in the `SYSTEM_DERIVED` namespace

is refused as a clean precondition no-op with the appended reason `SystemDerivedContentImmutable` (discriminant 12): the identifier is content-derived, and an in-place rewrite would silently invalidate its derivation (core specification, Chapter 5 system-derived identity). The same precondition applies to `ModifyIdentifiedPitch` (Section 3.5). The sanctioned path is minting a replacement pitch.

Conflict cases. Two concurrent modifications of one event with *differing* values produce a `StructuralFieldCollision` on the field `event`, recording the winner (later in canonical order) and the loser.

Undo semantics. Value-restoring (Section 3.34): undoing the enclosing transaction restores the event's chain-predecessor value, conflicting (`StrictInverse`) or skipping (`BestEffort`) when a later modification has superseded it. The restored value reverts the undone transaction's modifications and no more (since version 0.19.0): the pitches they removed come back, at the values they had and with their attachments; every live pitch of the event the value does not carry is kept, so a pitch another operation added since stays; and none a delete removed is revived, the undone transaction's own pitches being tombstoned first.

Re-anchoring. If the target event is tombstoned, the modification is a no-op (`TargetTombstoned/TargetMissing`

3.5 Identified-Pitch Operations

Payload schema. `InsertIdentifiedPitchOp { event: EventId, pitch: IdentifiedPitch }` mints a pitch into a live event; `DeleteIdentifiedPitchOp { pitch: PitchId }` tombstones one; `ModifyIdentifiedPitchOp { pitch: PitchId, value: Pitch }` overwrites a pitch's acoustic / scale-position value (distinct from `RespellPitch`, which overwrites only the *spelling*).

Canonical encoding. Insert: `event`, then the length-framed `pitch` value. Delete: `pitch`. Modify: `pitch`, then the length-framed `value`.

Reduction rule. The pitch-level analogues of the event-level mint, delete, and field overwrite, inheriting their disciplines (Sections 3.1, 3.2, and this chapter's field-overwrite treatment). **A note and a rest are the same slot under pitch add/remove** (normative): deleting the *only* pitch of a single-pitch note degrades the event to a `Rest` of the same id/voice/position/duration rather than leaving an empty pitched event (Chapter 5 forbids the empty chord, `ArenaError::EmptyPitchedEve` inserting a pitch into a rest is the dual, promoting it to a one-pitch note, and so is inserting one into an event of any other kind without a pitch list, an unpitched, indeterminate, graphic or cue event or a trajectory, which becomes a note of its own pitches (a trajectory's) and the new one (since version 0.19.0, before which the graph dropped the pitch the bookkeeping minted for every kind but a rest). Every pitch an event holds, whatever its kind (a trajectory's endpoints and steps), is read and written as a note's: a modification, transposition or deletion reaches it there, and a trajectory that loses one of its pitches becomes a note of the others, or a rest when none remain (since version 0.19.0, before which graph-aware reduction read and wrote a note's pitches alone). This preserves the delete-wins / mint disciplines and keeps the graph consistent with the bookkeeping, which tombstones or mints the pitch object either way. A grace note keeps a pitch, since without one it would be a rest of no length (since version 0.21.0): a delete of its last live pitch is refused `EventDurationInvalid`, as is a whole-event modification that would leave it none (Section 3.4); an undo taking its last, the grace staying, conflicts under `StrictInverse` and keeps the pitch under `BestEffort`, read as the undo leaves

the event, a pitch the undo brings back counting. A delete of the grace removes it.

Conflict cases. Insert and delete: none (mint is set-union; delete-wins is idempotent). Modify: two concurrent differing writes of one pitch produce a `StructuralFieldCollision` on the field `pitch`.

Undo semantics. Insert: undoing the enclosing transaction tombstones the minted pitch, re-resting the event if it was the only one (Section 3.34). Modify: value-restoring (Section 3.34) — the pitch's chain-predecessor value is restored unless superseded. Delete: still not inverted (re-introducing a tombstoned pitch is the deferred resurrection case, P11-C8).

Re-anchoring. An operation whose target event or pitch is tombstoned is a no-op; tombstoning a pitch runs the cross-cutting re-anchoring table over any structure that referenced it (see DeleteEvent).

3.6 Transpose (frozen; replay only)

Requirement 3.1

`Transpose` is **frozen**. Its reduction rule is fixed at the semantics described in this section, including the saturation and the duplicate-sensitivity, for as long as any stored operation carries it. A conforming implementation **MUST** reduce it exactly as specified here and **MUST NOT** emit it from new authoring; new transpositions are authored as `TransposeInterval` (Section 3.7).

Payload schema. `TransposeOp { targets: Vec<PitchId>, chromatic_steps: i32 }`. Pitch identifiers are preserved. The operation rewrites *scale position* (the `Cmn` alteration), not acoustic content.

Canonical encoding. `targets` in ascending canonical-byte order, then `chromatic_steps` as a little-endian `i32`. The sequence is *sorted but not deduplicated*: it is a multiset on the wire.

Reduction rule. An order-dependent content overwrite. For each live, non-system target *occurrence* in `targets`, the target's `Cmn alteration` is shifted by `chromatic_steps` and clamped to the `i8` bound. Consequences, all normative for replay:

- A target appearing k times is shifted k times.
- The nominal and octave never change: transposing C4 by $+12$ yields C4 with `alteration` $= 12$, not C5.
- A shift past the `i8` bound saturates silently, and the operation still reports `Applied`. Transposition under this rule is therefore not invertible.
- A target whose position is not `Cmn` is left unchanged, and the operation still reports `Applied`.

Rationale

Every bullet above is a defect (P12-K2), and none of them may be repaired in place. An operation is history: a replica replaying a stored `Transpose` must reconstruct the state

its author saw, and a "corrected" reduction rule would silently rewrite every score that ever used one. The defects are therefore *pinned*, not fixed, and the repair ships as a new operation kind whose discriminant no historical operation carries. Ratified Push 4a; supersedes the Pass 12 pin, which anticipated a payload schema-major and a dependency on the tuning catalog. Neither proved necessary: transposition acts on scale position, and appending an operation kind is a schema *minor* (Binary Format companion, requirement `req:binfmt:kind-discriminants`).

Conflict cases. None — composition is deterministic in canonical order (a deterministic repair, not a conflict).

Undo semantics. Transpose mints nothing, so the prototype's minted-object undo (Section 3.34) does not negate it, and it records no write into the pitch's value chain, so *value-restoring undo does not restore it either*: undoing a transaction containing a `Transpose` leaves the shifted pitch shifted. An inverse-interval undo does not exist (under saturation the inverse is not a function) and remains a deferred refinement (P11-C8).

This too is frozen. Making `Transpose` record into the value chain would not change its own reduction rule, but it *would* change what a stored history {`Transpose`, `UndoTransaction`} replays to — from "the pitch stays shifted" to "the pitch returns" — which is a change in what an existing document means. `TransposeInterval` records its write and *is* undoable (Section 3.7); the frozen operation is permanently not. That asymmetry is another reason never to author it.

Re-anchoring. Tombstoned targets are skipped (the transpose applies only to live pitches). `SYSTEM_DERIVED`-namespace targets are likewise *skipped* (ratified Pass 12, P12-K3): their intrinsic content is immutable, and an in-place alteration shift would desynchronize the content from the id's derivation inputs. A transpose whose live targets are *all* system-derived reduces as a precondition no-op (`SystemDerivedContentImmutable`).

3.7 TransposeInterval

Payload schema. `TransposeIntervalOp { targets: CanonicalSet<PitchId>, interval: TranspositionInterval }`, reusing the diatonic/chromatic pair the core specification defines in Chapter 2, Transposition and the Interval Type — the same value an `Instrument` carries as its written-versus-sounding interval. Pitch identifiers are preserved; the operation rewrites scale position and leaves acoustic realization untouched.

Requirement 3.2

`targets` is a **set**. It **MUST** be encoded as a sequence in ascending canonical-byte order with *strictly increasing* elements; a decoder **MUST** reject a sequence containing a duplicate rather than normalize it. A transposition is a function of *which* pitches it names, never of how many times it names them.

Canonical encoding. The strictly-increasing `targets` sequence, then `interval.diatonic_steps` and `interval.chromatic_steps`, each a little-endian `i32`.

Reduction rule. An order-dependent content overwrite. Tombstoned and `SYSTEM_DERIVED`

targets are skipped, exactly as for `Transpose`; if no mutable target remains the operation reduces as the corresponding precondition no-op. Otherwise, every remaining target **MUST** be transposable under core specification requirement `req:pitch:transposition`, and each is transposed by `interval`.

REQUIREMENT 3.3

If any mutable target is *not* transposable, the operation **MUST** reduce to a no-op that changes no pitch, reporting the precondition failure of the first such target in canonical order. It **MUST NOT** transpose the remaining targets. The four cases and their `PreconditionFailureReason` discriminants:

- a non-`Cmn` `scale_position.position` ⇒ `PitchSpaceMismatch` (6);
- a `Cmn` position whose enclosing chromatic structure or nominal mapping cannot be established ⇒ `PitchSpaceMismatch` (6);
- an `AcousticRealization::AbsoluteHz` realization ⇒ `AcousticRealizationPinned` (14);
- an `alteration`′ or `octave`′ that does not fit its field ⇒ `TranspositionOutOfRange` (15).

REQUIREMENT 3.4

A `TransposeInterval` **MUST** record the spelling its interval determined as a `SpellingAttachment` with source `SpellingSource::Propagated`, per the core specification's Chapter 2, SPELLING SOURCES. The attachment's `from` is the transposed pitch's own identifier, which the operation preserves.
That attachment alone is *not sufficient*, because the default precedence ranks `UserChosen` and `Imported` above `Propagated` (Chapter 2, CONFIGURABLE PRECEDENCE). A `TransposeInterval` **MUST** therefore also **move** every engraved-layer, pitch-scoped, explicit spelling attachment on each target that is not itself `Propagated`, preserving each attachment's `source`, `priority`, and `layer`. A spelling is moved by transposing its *nominal* diatonically and taking whatever accidental the transposed pitch then requires at that staff position. A quarter-tone's spelling (a pitch in `cmn-24`) takes it in quarter-tones: a whole number of semitones takes the standard accidentals, and an odd number of quarter-tones the quarter-tone accidental of the spelling's own kind, an arrowed accidental keeping its arrow and Stein's staying Stein's where it has one for the alteration (since version 0.19.0; before it, any authored spelling on a `cmn-24` pitch refused). An attachment that cannot be written at the transposed position (an octave out of range, or an alteration only a repeated accidental would write) **MUST** be removed, the pitch then taking the spelling the transposition propagates; before version 0.19.0 the operation refused instead, which only graph-aware reduction could see. A transposition whose transposed *value* no well-formed accidental stack writes — past a triple sharp or flat, or in `cmn-24` an odd alteration past five quarter-tones — **MUST** refuse, atomically, with `TranspositionOutOfRange`, in both modes, each reading the value.

Rationale

Two different failures hide here, and the propagated attachment addresses only the first. For a pitch with *no* authored spelling, the pre-pass would re-infer, and may pick the enharmonic the interval did not choose. The propagated attachment pins the interval's determination.

For a pitch *with* an authored spelling, the propagated attachment is outranked and changes nothing: a C4 the author spelled "C", sharpened to C-sharp 4, would still resolve to `Authored(UserChosen)` and engrave a C natural — the accidental simply disappears, and the notehead names a pitch that is not there. So the authored spelling is moved instead of being left or discarded. It is moved by its **nominal**, because the nominal is what carries the author's enharmonic decision: an author who wrote B-sharp 3 rather than C4 chose the letter B, and a perfect fifth up must give F-double-sharp 4, not G. Discarding the attachment would silently lose that decision; re-inferring it from the transposed pitch would give G.

A transposed `UserChosen` spelling is still the user's choice, so its source is preserved. `Imported` likewise: an imported attachment asserts "this pitch is written thus", and once the pitch moves that assertion must move with it or become false. Import fidelity is a property of the source file, which a transposition does not touch.

Conflict cases. None — composition is deterministic in canonical order.

Undo semantics. Nothing is minted. `TransposeInterval` **MUST** record each transposed pitch's new value into that pitch's value chain, and that pitch's resulting *engraved spelling set* into the spelling-set chain, so that value-restoring undo recovers the pre-transpose pitch *and* its pre-transpose spelling. Restoring the pitch alone would leave a notehead spelled for a pitch that no longer exists. The value chain is written in both reduction modes, base-free reduction reading each pitch its operation set minted at the value it holds (since version 0.19.0; before it, base-free reduction, holding no pitch values, recorded nothing). This is where `TransposeInterval` departs from the frozen `Transpose`, which records nothing and is therefore not undoable.

Requirement 3.5

The pitch's engraved spelling set is a single undo key with *more than one writer*. Every operation that changes it — `RespellPitch` and `TransposeInterval` alike — **MUST** record a write on that key. Consequently a `RespellPitch` causally prior to a transposed transaction is that transaction's chain-predecessor and is restored by its undo; and a `RespellPitch` canonically *later* supersedes the transaction's write, so a `StrictInverse` undo conflicts rather than erasing it, per the general superseded-writer rule.

A pitch's value and its engraved spelling set **MUST** undo as one unit: if either is superseded, neither is restored. A `BestEffort` undo **MUST NOT** restore a pitch's earlier value while leaving a spelling authored against the value it has since taken.

This unit is keyed on the *pitch*, and on which keys the *transaction* wrote — not on which operation wrote them, and not on `TransposeInterval` in particular. A transaction whose members write the two halves separately is coupled the same way: an editor's "move note" is a `ModifyIdentifiedPitch` (the value) together with a `RespellPitch` (the

spelling set), and a later respell makes a `BestEffort` undo skip both. That breadth is intended. A transaction that wrote only one of the two keys is unaffected, since an unwritten key yields no supersession.

Rationale

Ratified as P13-S3. An implementation may keep the spelling-set chain physically separate from whatever chain drives `RespellPitch`'s last-writer-wins conflict detection — the reference implementation does, because folding transposes into that chain would make a concurrent respell *conflict* with a transpose and would move the canonical bytes of every existing history. What it may not do is let one writer own the key. A chain with a single writer cannot see the other's prior value (so undo erases it) and cannot see the other's later value (so undo overwrites it).

An inverse interval usually exists ($(-d, -c)$), because the reduction never saturates — but not always: `i32::MIN` has no negation. Undo does not rely on it.

Re-anchoring. As `Transpose`.

3.8 CreateCrossCutting

Payload schema. `CreateCrossCuttingOp { structure: CrossCuttingValue }`, where `CrossCuttingValue` is the typed value of a `Tie`, `Slur`, `Beam`, `Spanner`, or, since version 0.21.0, a `Marker` (a point mark: a dynamic, fermata, breath mark, caesura, staff text, tempo mark, rehearsal mark, segno or coda) or a `Lyric` (one syllable on an event). Its identity and referenced endpoints are read from the value.

Canonical encoding. A discriminant for the structure kind (`0` tie, `1` slur, `2` beam, `3` spanner, `4` marker, `5` lyric), then the length-framed canonical bytes of the structure value. A value holding a marker, a lyric, a wavy line or a pedal line without its sign is stamped schema major 5; any other is stamped major 2, as before.

Reduction rule. Set-union creation: the structure is minted live if its id is not already live and every referenced endpoint is live; a second create of a live id is idempotent. A spanner's staves are referents, each checked in both modes as Requirement 2.2 reads one (since version 0.19.0), a dead staff refusing the create, as it does a `ModifyCrossCutting`, with `TargetMissing`. Graph-aware reduction materialises the structure with its full fields. An event carries one syllable per verse: a create or a modification of a lyric whose event and verse a live lyric already holds is refused with `SlotOccupied`, in both modes, and of two concurrent ones the later in canonical order is refused (since version 0.21.0). A marker or spanner anchored to a region by a musical offset is held to the region's time, as a clef or key change is: a create or modification placing one in a region that admits no musical offset is refused `WrongRegionTimeModel`, and a live one strands its region's migration out of musical time, the conflict naming it (since version 0.21.0, before which the value applied there or stayed, breaking `AnchorOffsetModel`).

Conflict cases. None (all-or-nothing creation; union is deterministic).

Undo semantics. Undo tombstones the minted structure, under the active policy.

Re-anchoring. The structure participates in the re-anchoring rule table when one of its endpoints is later tombstoned (see DeleteEvent). A lyric cascade-deletes with its event; a marker anchored to an event re-anchors to the nearest live event of its staff instance, or orphans to its region's start, in both modes, base-free reduction reading the event's place from the occupancy index (since version 0.21.0). The repaired anchor is the one every later verdict reads: an orphan at its region's start holds the region as any marker anchored there does, against a `DeleteRegion` and an undo of the region's create (since version 0.21.0, after its first review, before which the repair moved the graph's anchor alone and the region could be removed from under the marker). An undo that removes the marker's event reads the marker as the repair will leave it, at the nearest event of its staff instance the undo leaves or else at the start of the event's region, so a marker the undo would orphan holds that region against the same undo (since version 0.21.0, after its second review, before which the undo read the marker at its event and removed the region with the event, the orphan left anchored to it). Both readings take the marker as the whole undo leaves it, its restoration included: a marker the undone transaction itself moved onto a note the undo removes, or onto the start of a region it removes, returns to its earlier anchor and holds nothing, so the undo applies (since version 0.21.0, after its third review, before which the undo read such a marker where the transaction had put it: a strict or cascading undo conflicted, and a best-effort one kept the emptied region).

A tie gives way (since version 0.19.0). A tie holds only while its class's validation holds (core specification, `req:graph:tie-class-validation`): its pairing, and its ends' places in their voices. An operation that moves a pitch or an event is not refused for breaking one; the tie gives way.

Requirement 3.6

When a transaction completes, a lone operation being its own transaction, each live tie its members may have broken **MUST** be held to its class's validation, read in both modes from the indices both keep (each event's voice and position, its live pitches and their values); one that no longer holds **MUST** be removed as a `DeleteCrossCutting` removes it, and the effect of the last member that touched it **MUST** record a `CascadeDeleted` repair targeting it. A tie one member breaks and a later member of its transaction mends stands. The operations so held are an insert (its new neighbours), a whole-event modify (the event and its new neighbours), every pitch operation and transposition (the pitches' events), a migration and an undo (every live tie), and a create or modify of the tie itself, which so gives way, when its transaction completes, to a change it was concurrent with. A conflicted operation's effect carries no repair, and there the tie's tombstone, which names the operation, is the record.

Rationale

Before version 0.19.0 a tie was accepted as written and never read again, so a concurrent insert between its ends, or a transpose of one end, left the graph breaking `TiePairing`. Refusing such an edit would let a tie block every edit near it; a tie is notation of a pairing, and when the pairing goes, so does the tie. A transaction is one edit from its author's view, so its tie is held when it completes, not after each member: moving a tied pair one note at a time keeps the tie.

Authoring advisory (ratified Pass 12, closing P12-K12). The slur-spanning advisory reads `Region.permits_spanning_slurs` *conjunctively*: a slur (or other spanner) whose endpoints lie in different regions passes the advisory only when *both* endpoint regions permit spanning. The check is authoring-time only and never alters reduction.

Migration coverage. The v0→v1 migration (Chapter 4) reconstructs the event-anchored `Tie`, `Slur`, and `Beam` from the v0 reference (id plus event endpoints). A `Spanner` is anchored by `TimeAnchor`s rather than a fixed pair of event endpoints, so its full value is not reconstructable from the v0 event-reference projection; a `Spanner`-create is therefore reported unmigratable (read-only), alongside the respell case of P12-K1, and remains so. A faithful spanner migration awaits a richer v0 projection that carries the anchors — a Phase-3 / Pass-12 extension, not yet implemented.

3.9 DeleteCrossCutting

Payload schema. `DeleteCrossCuttingOp { structure: TypedObjectId }` — the structure named by the same key the set-union creation and the re-anchoring table use; it **MUST** be a cross-cutting kind (`Tie/Slur/Beam/Spanner/Marker/LyricLine`).

Canonical encoding. The canonical bytes of `structure`.

Reduction rule. Delete-wins: the structure is tombstoned (its identifier retained). A second delete of the same structure is idempotent; deleting a missing or non-cross-cutting id is a no-op precondition failure. Graph-aware reduction removes the structure from the score.

Conflict cases. None (delete-wins is idempotent).

Undo semantics. A delete mints nothing, so the prototype's minted-object undo (Section 3.34) does not re-introduce the tombstoned structure (P11-C8).

Re-anchoring. The deletion is direct (the structure is the target, not a referenced endpoint); it does not itself trigger the endpoint re-anchoring table.

3.10 ModifyCrossCutting

Payload schema. `ModifyCrossCuttingOp { structure: CrossCuttingValue }` — the full replacement value (v1). The replacement keeps the structure's identity but may change its endpoints and per-kind fields; the LWW key is the structure's `CrossCuttingValue::id`.

Canonical encoding. The discriminant and length-framed bytes of the `CrossCuttingValue` (as for CreateCrossCutting).

Reduction rule. A last-writer-wins field overwrite keyed by structure id. Precondition: the structure is live. The reduction re-derives the structure's endpoints from the new value, so a later re-anchoring sees them. A malformed replacement is a precondition no-op — in particular a beam whose membership falls below the two-member minimum is refused rather than materialised. Graph-aware reduction overwrites the structure in place.

Conflict cases. Two concurrent differing modifications of one structure produce a `StructuralFieldCollision` on the field `cross_cutting`.

Undo semantics. Value-restoring (Section 3.34): undo restores the structure's chain-predecessor value unless superseded. A lyric restored to an event and verse another live syllable holds, as the undo leaves it, is superseded by the write that put that syllable there, so `StrictInverse` conflicts and `BestEffort` leaves the syllable where it stands; syllables one undo moves back among themselves restore together (since version 0.21.0, after its first review, before which the undo restored a second syllable into the slot).

Re-anchoring. If the target structure is tombstoned, the modification is a no-op; re-deriving the endpoints lets a subsequent endpoint tombstone re-anchor the structure through the standard table (see DeleteEvent).

3.11 ChangeRegionTimeModel

Payload schema. `ChangeRegionTimeModelOp { region: RegionId, new_time_model: RegionTimeModel, declared_incompatible: Vec<EventId>, remapping: PositionRemapping }` — the full target model value (v1).

Canonical encoding. `region`, the length-framed `new_time_model` value, the canonically-ordered `declared_incompatible` set, then `remapping`.

Reduction rule. Structural migration. The region adopts the target time model. Both reduction modes derive the coordinate-kind incompatibilities from the reducer's graph-independent indices, so they agree: the region's events are those with a metric placement in the occupancy index, found through its staff instances and voices (a voice promoted during reduction through the insert it was promoted for), and each is incompatible with a target whose coordinate discipline does not admit an event in musical time, read as invariant 4 reads an event's coordinates (a proportional target, or an aleatoric one anchored in wall-clock time; since version 0.19.0, before which only a proportional target was), and with any target when a `Reassign` leaves it unmapped, or would leave it overlapping another event of its voice (invariant 3; since version 0.19.0, the migration having applied before it). A target that admits no musical offset strands each of the region's live measures as well, named beside the events, and a live tempo segment of any map anchored to the region by a musical offset strands the region itself, named among them, the segment having no identifier, and a live staff instance of the region holding a clef or key change, which stands at a musical offset, strands that instance, named among them, and an applied such migration drops the region's system and page breaks and its default and local metric grids (since version 0.19.0). Graph-aware reduction also judges, from the graph, the region's events the index does not hold (a base's events of another coordinate kind, which base-free reduction never has), against the target's discipline as invariant 4 does (an aleatoric target by its anchoring discipline since version 0.19.0, before which it was taken to admit every event). A migration with incompatible events is refused as below; an applied `Reassign` moves the occupancy index in both modes, and the region's new coordinate discipline is recorded in both, so a later insert, move or re-anchoring reads the same state in each. Graph-aware reduction keeps each voice whose events a `Reassign` moves in position order, as a `ModifyEvent` move does (since version 0.19.0, before which the voice kept its old order). That the region is live is a referential precondition, checked in both modes as Requirement 2.2 reads a referent (since version 0.19.0; graph-aware only before it).

Conflict cases. Concurrent same-region migrations produce a `StructuralFieldCollision` on the field `time_model`; a migration with incompatible events produces a `TimeModelMigrationFailure`

naming the region and the incompatible events. A causally-later migration is re-evaluated against the first migration's graph rather than conflicting.

Undo semantics. Undo restores the region's prior time model under the active policy.

Re-anchoring. Not applicable.

Open Question

P11-C6. The rich migration payload — a coordinate converter rather than a `declared_incompatible` list plus a `PositionRemapping` — remains the catalog's to design when graph-aware migration is the only reduction path.

3.12 Structural Containers

Payload schema. Three create/delete pairs over the region hierarchy: `CreateRegionOp { region: Region } / DeleteRegionOp { region: RegionId }; CreateStaffInstanceOp { region: RegionId, instance: StaffInstance } / DeleteStaffInstanceOp { staff_instance: StaffInstanceId }; CreateVoiceOp { staff_instance: StaffInstanceId, voice: Voice } / DeleteVoiceOp { voice: VoiceId }`. Each create carries the full container value (v1); the reduction preconditions it bears *no typed child object* — an empty container.

Canonical encoding. Create: the parent id (where the schema names one), then the length-framed canonical bytes of the container value. Delete: the container id.

Reduction rule. Set-union creation of an *empty* container, and an *empty-only* delete-wins tombstone. A create mints the container live if its id is fresh and (for staff instance and voice) its parent is live; it preconditions the carried value to bear no typed child object (a region: no staff instances, barline-alignment groups, or graphic objects; a staff instance: no voices or measures; a voice: no events), since those carry distinct `TypedObjectId`s the reducer mints separately — so contents are added by subsequent operations. A delete is a delete-wins tombstone, but a *precondition no-op* (`ContainerNotEmpty`) unless the container has no live children — the caller deletes contents first. A region is held so by a live tempo segment of another map anchored to it and, since version 0.21.0, by a live marker or spanner anchored to it. Graph-aware reduction adds or removes the container and maintains the region's staff extent so `RegionExtents` stays satisfied.

Conflict cases. None at reduction time: creation is set-union (a repeat create is idempotent), and the empty-only delete is a deterministic precondition gate, not a conflict.

Undo semantics. Undo of a *create* tombstones the minted container (Section 3.34); `StrictInverse` conflicts if it was concurrently mutated, with the policy treatment as for InsertEvent, and, since version 0.19.0, if a live child the undo leaves is in it (a region's staff instance, an instance's voice, a voice's event), which it would leave naming nothing, or a live tempo segment of another map is anchored to a region it would remove, as a `DeleteRegion` is refused (the conflict naming the region, the segment having no identifier), or, since version 0.21.0, a live marker or spanner the undo leaves is anchored to it, read as the undo's restorations leave it (the conflict naming the structure). An undone region or staff instance leaves the graph-aware score, as a deleted one does, and an undone staff instance or voice leaves its container's

place in the reducer's indices, so a region or instance the undo emptied is empty to a later `DeleteRegion` or `DeleteStaffInstance` (since version 0.19.0, before which the delete was refused `ContainerNotEmpty`). A *delete* mints nothing, so the prototype's minted-object undo does not re-introduce it (P11-C8).

Re-anchoring. Not applicable (the containers are minted/tombstoned by id; the empty-only precondition means a delete never strands live children).

3.13 CreateStaff

Payload schema. `CreateStaffOp { staff: Staff }` — the full global-staff value (v1): identity, name, abbreviation, instrument reference, default staff-line configuration, and optional group membership.

Canonical encoding. The length-framed canonical bytes of `staff`.

Reduction rule. Set-union creation of a global `Staff` on the score root, completing the structural-container family (Section 3.12) upward: staff *instances* reference global staves, and until this primitive existed a resolvable staff could only be base-seeded. A create mints the staff live if its id is fresh; a repeat create carrying a byte-identical value reduces idempotently, and a create whose id is already live with a *differing* value is a precondition no-op with the appended reason `RecreateContentMismatch` (discriminant 13; ratified Pass 12, closing P12-K9 — the former `TargetMissing` reuse misnamed the situation: the target is not missing, its content disagrees). The same reason applies to the carried `TimeSignature` (Section 3.29), the other re-create site at which the reducer retains the carried value for comparison. The structural-container creates (Section 3.12) are plain set-union — any repeat create of a live id reads `AlreadyApplied` without value comparison, since the carried value is preconditioned empty of children. Graph-aware reduction additionally preconditions that the referenced instrument is live and, when `group` is present, that the staff group resolves — the mint must leave the graph satisfying the reference-resolution invariants.

With staves mintable, `CreateStaffInstance` (Section 3.12) preconditions that the instance's referenced `Staff` is live (previously the reference was satisfiable only from the seeded base, so the check was vacuous), and that no live instance of the region already manifests it: a region manifests a staff once (graph invariant `StaffInstanceResolves`), so a second is refused with `ContainerNotEmpty`, the region's place for the staff being taken (since version 0.19.0).

Membership maintenance (P13-S16, disposition A; supersedes genesis tranche G3a's disposition B). `Staff.group` is the **sole authority** for group membership, and `StaffGroup.members` (Section 3.18) is a projection **maintained from it** under graph-aware reduction. `CreateStaff` carrying `group: Some(g)` therefore appends the newly minted staff to `g`'s `members` in the graph. The append is set-union and idempotent per staff id — convergence replays operations, so appending an id already present **MUST NOT** duplicate it — and base-free reduction has no graph to maintain and writes nothing. The two **MUST** agree in both directions, and graph invariant 21 (core specification GRAPH INVARIANTS) flags any disagreement between live objects.

What this replaced. Under disposition B, `CreateStaff` authored a **permitted missing-member** stale form: a `CreateStaffGroup(g, members: [])` followed by a `CreateStaff(s, group: Some(g))` left `s.group == Some(g)` while `g.members` omitted `s`, because that tranche minted only and

no operation maintained the projection. **That outcome is no longer reachable**: the identical sequence now yields agreement. The mirror-image **spurious** form is likewise unauthorable — `CreateStaffGroup` refuses a non-empty carried `members` outright; see Section 3.18.

Conflict cases. None at reduction time (set-union; the differing-value re-create is a precondition gate, not a conflict).

Undo semantics. Undo of a create tombstones the minted staff (Section 3.34); `StrictInverse` conflicts if a live staff instance references it (tombstoning it would strand the instance), and so, since version 0.19.0, if a live part definition or spanner names it, before which the undo removed the staff and left the part or spanner naming nothing. **Undo also strips the removed staff's id from every live group's `members` (P13-S16)**: the projection is maintained from `Staff.group`, so a staff leaving the graph **MUST** leave the projection with it. Without that strip the undo would leave a group naming a staff that no longer exists. Note this is the *unguarded* direction of the pair — undo of a `CreateStaffGroup` is refused while a live staff still names the group (Section 3.18), but nothing refuses undoing the staff itself.

Re-anchoring. Not applicable (a staff mint references no tombstonable anchor; there is no `DeleteStaff` in this catalogue revision — an empty-only staff delete mirroring the container discipline is a later schema-fill).

3.14 CreateInstrument

Ratified with the genesis operation tranche's first rung (G1; `spec/CONTRACT_GENESIS_G1_INSTRUMENT.md`, executing `spec/RULING_GENESIS_PERSISTENCE.md`). `CreateStaff` (Section 3.13) already requires a live `Instrument`, and until this primitive existed nothing could mint one except a canonical base — so a document created empty (`Score::empty`) and given only operations could reach a `CreateStaff` but never satisfy it. `CreateInstrument` is the single missing link between an empty score and a note: with it, the chain `CreateInstrument` → `CreateStaff` → `CreateRegion` → `CreateStaffInstance` → `CreateVoice` → `InsertEvent` materializes a note-bearing score from operations alone.

Payload schema. `CreateInstrumentOp { instrument: Instrument }` — the full abstract-instrument value (schema major 2): identity, name, declared pitch range, abbreviation, sound configuration, transposition interval, default clef, default staff-line configuration, and unpitched members. Unlike every carried type in Section 3.12 and Section 3.13, `Instrument` holds no outbound entity reference, so this primitive needs no referential precondition at all — only mint and idempotence.

Canonical encoding. The length-framed canonical bytes of `instrument`. `Instrument`'s schema-major-2 appends (`sound_config`, `default_clef`, `default_staff_lines`, and the rest) are mandatory fields, not `Option`-hidden, so no lower-major layout for this payload exists: under minimal stamping (Binary Format companion Schema Major 2) a block carrying a `CreateInstrument` is *born at v2*, exactly as `CreateStaff` is — not the value-dependent shape `CreateRegion`/`SetStaffLayout` use.

Reduction rule. Set-union creation of an `Instrument` on the score root, mirroring `CreateStaff`'s discipline exactly since neither carried type references another entity: a create mints the instrument live if its id is fresh; a repeat create carrying a byte-identical value reduces idempotently (`NoOpReason::AlreadyApplied`); a create whose id is already live with a *differing* value is a

precondition no-op with `RecreateContentMismatch`; and a create naming a tombstoned id is a precondition no-op with `TargetTombstoned`. There is no graph-aware reference-resolution block, because there is nothing for one to check.

Conflict cases. None at reduction time (set-union; the differing-value re-create is a precondition gate, not a conflict).

Undo semantics. Undo of a create tombstones the minted instrument; `StrictInverse` conflicts if a live staff references it (tombstoning it would strand the staff), and, since version 0.19.0, if a live staff instance's instrument override names it.

Re-anchoring. Not applicable, for the same reason as `CreateStaff`: an instrument mint references no tombstonable anchor. There is no `DeleteInstrument` in this catalogue revision — `CreateStaff` ships today with no `DeleteStaff` either, and full delete/modify coverage of the genesis-authored fields is later tranche work (`spec/PLAN_GENESIS_OPS.md` G3).

3.15 SetCanvasLayoutDefaults

Ratified with the genesis operation tranche's G2a rung (`spec/CONTRACT_ GENESIS_G2A_SETTINGS.md`, executing `spec/RULING_GENESIS_ PERSISTENCE.md`): the canvas's default layout advisories (`Canvas.layout_defaults`) join the operation-authored surfaces, on exactly the `SetMetadata` pattern (Section 3.28).

Payload schema. `SetCanvasLayoutDefaultsOp { layout_defaults: CanvasLayoutDefaults }` — the full page-size-and-margins value.

Canonical encoding. The length-framed canonical bytes of `layout_defaults`. `CanvasLayoutDefaults` carries no schema-major-1-or-higher appends of its own — the versioning that labels the type "schema major 1" lives in the *containing* `Canvas` walk, not in this leaf — so, as a standalone payload, it has exactly one layout and this operation is schema major 0 unconditionally.

Reduction rule. An advisory last-writer-wins field overwrite, identical in discipline to `SetMetadata`: the latest write in canonical order silently wins and the operation always applies — no working state, no conflict, and no idempotence short-circuit (a re-write of an identical value is a legitimate new write, not a no-op). Graph-aware reduction overwrites `Canvas.layout_defaults`.

Conflict cases. None (advisory LWW).

Undo semantics. A value-restoring field overwrite (Section 3.34): the write chain is seeded from the base value at ingest, so undoing the first operational write restores the pre-operational state rather than absence.

Re-anchoring. Not applicable: the field is a score singleton with no tombstonable anchor.

3.16 SetSpellingPrecedence

Ratified with the genesis operation tranche's G2a rung, alongside `SetCanvasLayoutDefaults` above: the score's spelling-precedence configuration (`Score.spelling_precedence`) joins the operation-authored surfaces, on the same `SetMetadata` pattern.

Payload schema. `SetSpellingPrecedenceOp { precedence: SpellingPrecedence }` — the full total-ordering value over the five spelling-source kinds.

Canonical encoding. The length-framed canonical bytes of `precedence`. `SpellingPrecedence` has never been versioned — every codec walk that reads it, frozen and live alike, reads it through the plain unversioned codec — so this operation is schema major 0 unconditionally, exactly as `SetCanvasLayoutDefaults` is.

Reduction rule. An advisory last-writer-wins field overwrite, identical in discipline to `SetMetadata` and `SetCanvasLayoutDefaults`: no working state, no conflict, no idempotence short-circuit. Graph-aware reduction overwrites `Score.spelling_precedence`.

Conflict cases. None (advisory LWW).

Undo semantics. A value-restoring field overwrite (Section 3.34): the write chain is seeded from the base value at ingest, so undoing the first operational write restores the pre-operational state rather than absence.

Re-anchoring. Not applicable: the field is a score singleton with no tombstonable anchor.

3.17 SetTuningContext

Ratified with the genesis operation tranche's G2b rung (`spec/ CONTRACT_GENESIS_G2B_TUNING.md`, executing `spec/ RULING_GENESIS_PERSISTENCE.md`): the score's tuning context settings (`Score.tuning_conte` join the operation-authored surfaces, on the same `SetMetadata` pattern as `SetCanvasLayoutDefaults` and `SetSpellingPrecedence` above. Unlike either sibling, `SetTuningContext` is the **sole genesis payload born at schema major 3** and is the surface that raises the `OperationEnvelopeBlock` accept-set from `[0, 2]` to `[0, 3]` (Binary Format companion The Chunk-Level Gate).

Payload schema. `SetTuningContextOp { settings: TuningContextSettings }`. `TuningContextSettings` is the **authored subset** of `ScoreTuningContext`: exactly its five wire-bearing fields (`default_pitch_space`, `default_tuning_system`, `reference`, `smufl`, `overrides`), in the codec's existing order — not the full six-field graph type. `accidental_extensions` is deliberately **not** a field of the carried payload: it is staged out of schema major 3 (Binary Format companion Schema Major 3) and `OperationSet::accept` stores the authored envelope as a *value*, so a payload carrying the full `ScoreTuningContext` would let `accidental_extensions` reach the graph on the authoring replica while decoding to empty on every replica that received the document through serialization — the same document in two graph states, depending only on whether it was just authored or just reloaded. The subset type makes this divergence unrepresentable: a field the payload does not carry cannot be set inconsistently.

Canonical encoding. The length-framed canonical bytes of `settings`. `TuningContextSettings`'s encoding is **byte-identical** to `ScoreTuningContext`'s existing five-field wire walk — a type-level narrowing, not a new wire form. Since `ScoreTuningContext`'s `smufl` and `overrides` appends are mandatory (not `Option`-hidden), no lower-major layout for this payload exists at any value: `SetTuningContext` is **born at schema major 3 unconditionally**, the same shape as `CreateInstrument`'s unconditional birth at major 2 (Section 3.14), not the value-dependent shape `CreateRegion`/`SetStaffLayout` use.

Reduction rule. An advisory last-writer-wins field overwrite, identical in discipline to `SetMetadata`: no working state, no conflict, and no idempotence short-circuit. Graph-aware reduction writes

exactly the five subset fields onto `Score.tuning_context` and **leaves `accidental_extensions` untouched** — preserving whatever the graph already held, since the operation carries no information about that field and normalizing it away (clearing it at construction, or rejecting a populated in-memory value) would either make correctness depend on a clearing discipline the compiler cannot see, or turn an in-memory-only field into an authoring error for callers who never opted into its persistence.

Conflict cases. None (advisory LWW).

Undo semantics. A value-restoring field overwrite (Section 3.34), with the chain's value type the **subset** `TuningContextSettings`, not `ScoreTuningContext`: the write chain is seeded from the base's five wire-bearing fields at ingest, so undoing the first operational write restores the pre-operational settings rather than absence, identically whether the base was seeded with the type default or a genuine non-default value — the never-authored / authored-to-default distinction is unobservable and stays so, since `ScoreTuningContext` is an always-valued `Score` field, like `metadata`, with no absent state to return to. `accidental_ extensions` never participates in undo: no operation ever writes it, so undo leaves it exactly as it found it.

Re-anchoring. Not applicable: the field is a score singleton with no tombstonable anchor.

3.18 CreateStaffGroup

Ratified with the genesis operation tranche's G3a rung (`spec/ CONTRACT_GENESIS_G3A_ENTITIES.md`, executing `spec/RULING_ GENESIS_PERSISTENCE.md`): a global staff grouping (`Score. staff_groups`) joins the operation-authored surfaces, on the `CreateStaff` set-union mint pattern (Section 3.13).

Payload schema. `CreateStaffGroupOp { group: StaffGroup }` — the full staff-group value: identity, optional name, kind (grand staff, bracket, sub-bracket, choral, or a registered extension kind), and the carried `members` list.

Canonical encoding. The length-framed canonical bytes of `group. StaffGroup` has never been versioned, so this operation is schema major 0 unconditionally.

Reduction rule. Set-union creation, mirroring `CreateStaff` exactly: a create mints the group live if its id is fresh; a repeat create carrying a byte-identical value reduces idempotently (`NoOpReason::AlreadyApplied`); a create whose id is already live with a differing value is a precondition no-op with `RecreateContentMismatch`; a create naming a tombstoned id is a precondition no-op with `TargetTombstoned`.

A non-empty carried `members` is refused (P13-S16). This operation authors the group, not its membership, so reduction preconditions that the carried `members` is **empty**, refusing with `ContainerNotEmpty` (discriminant 10) — no new `PreconditionFailureReason`. **This precondition is not graph-gated**: it asks only about the carried value, so unlike the sibling mints' member-liveness checks it holds **base-free as well**. Membership is maintained from `Staff.group` instead (Section 3.13).

This replaced a graph-aware liveness precondition. Under disposition B this operation accepted a carried `members` and graph-aware reduction additionally preconditioned that every id in it resolved to a live `Staff`, reusing `TargetMissing` (discriminant 0) and skipped entirely base-free. Both halves are gone: liveness is never consulted, because the container must be empty before the question could arise.

Membership maintenance (P13-S16, disposition A; supersedes disposition B). `Staff.group` (Section 3.13) is the **sole authority** for group membership, and `members` here is a projection **maintained from it** by reduction. The two **MUST** agree in both directions; graph invariant 21 (core specification GRAPH INVARIANTS) flags any disagreement between live objects, and no consumer may read `members` to decide membership in preference to `Staff.group`.

What this replaced. Under disposition B, `members` was stored exactly as carried and **neither maintained nor trusted**, and this operation authored a **permitted spurious-member** stale form: a `CreateStaff(s, group: None)` followed by `CreateStaffGroup(g, members: [s])` left `g.members == [s]` while `s.group` stayed `None` (or named a different group). **That sequence now refuses** the second operation with `ContainerNotEmpty`, minting no group at all, so the form is unauthorable rather than merely discouraged.

Conflict cases. None at reduction time (set-union; the differing-value re-create is a precondition gate, not a conflict).

Undo semantics. Undo of a create tombstones the minted group (Section 3.34); `StrictInverse` conflicts if a live staff still names it (a stale reference, not a strand — `Staff.group` degrading to point at a tombstoned id is the same shape core Chapter 6's re-anchoring rule table already treats for other dangling references).

Re-anchoring. Not applicable (a staff-group mint references no tombstonable anchor; there is no `DeleteStaffGroup` in this catalogue revision, mirroring `CreateStaff`'s own absent delete).

3.19 CreatePartDefinition

Ratified with genesis tranche G3a, alongside `CreateStaffGroup` above: a part-extraction view definition (`Score.parts`) joins the operation-authored surfaces, on the same `CreateStaff` set-union mint pattern.

Payload schema. `CreatePartDefinitionOp { part: PartDefinition }` — identity, name, and the carried `staves` list. Parts are projections, not storage (core specification Chapter 5 PARTS): only references and overrides.

Canonical encoding. The length-framed canonical bytes of `part. PartDefinition` has never been versioned, so this operation is schema major 0 unconditionally.

Reduction rule. Set-union creation, mirroring `CreateStaff` exactly (mint/idempotence/mismatch/tombstone as above). Graph-aware reduction additionally preconditions that every id in `staves` resolves to a live `Staff` (`TargetMissing`, no new failure reason), skipped base-free.

Conflict cases. None at reduction time (set-union; the differing-value re-create is a precondition gate, not a conflict).

Undo semantics. Undo of a create tombstones the minted part (Section 3.34); `StrictInverse` conflicts if concurrently mutated.

Re-anchoring. Not applicable (a part mint references no tombstonable anchor; there is no `DeletePartDefinition` in this catalogue revision).

3.20 CreateAnalysisLayer

Ratified with genesis tranche G3a: a first-class analysis layer (`Score.analysis_layers`) joins the operation-authored surfaces, on the `CreateInstrument` mint pattern (Section 3.14) — not `CreateStaff`'s, since `AnalysisLayer` holds no outbound entity reference.

Payload schema. `CreateAnalysisLayerOp { layer: AnalysisLayer }` — identity and name only (core specification Chapter 5 Analysis Layers and Views).

Canonical encoding. The length-framed canonical bytes of `layer. AnalysisLayer` has never been versioned, so this operation is schema major 0 unconditionally.

Reduction rule. Set-union creation, mirroring `CreateInstrument` exactly: mint/idempotence/mismatch/tombs as above, with **no** graph-aware reference-resolution block — there is nothing for one to check.

Conflict cases. None at reduction time (set-union; the differing-value re-create is a precondition gate, not a conflict).

Undo semantics. Undo of a create tombstones the minted layer (Section 3.34); `StrictInverse` conflicts if a live view still names it in `active_layers` (Section 3.21).

Re-anchoring. Not applicable (an analysis-layer mint references no tombstonable anchor; there is no `DeleteAnalysisLayer` in this catalogue revision).

3.21 CreateView

Ratified with genesis tranche G3a, the fourth and final root-level mint of this rung: a view recipe (`Score.views`) joins the operation-authored surfaces, on the `CreateStaff` set-union mint pattern.

Payload schema. `CreateViewOp { view: ViewDefinition }` — identity, name, and the carried `active_layers` list. The view-*kind* detail (core specification Chapter 7) is out of scope for this baseline.

Canonical encoding. The length-framed canonical bytes of `view. ViewDefinition` has never been versioned, so this operation is schema major 0 unconditionally.

Reduction rule. Set-union creation, mirroring `CreateStaff` exactly (mint/idempotence/mismatch/tombstone as above). Graph-aware reduction additionally preconditions that every id in `active_layers` resolves to a live `AnalysisLayer` (`TargetMissing`, no new failure reason), skipped base-free. **This packet is self-contained**: a `CreateView`'s precondition target can be minted by a `CreateAnalysisLayer` earlier in the same operation set, so the ordering is testable end-to-end without a canonical base.

Conflict cases. None at reduction time (set-union; the differing-value re-create is a precondition gate, not a conflict).

Undo semantics. Undo of a create tombstones the minted view (Section 3.34); `StrictInverse` conflicts if concurrently mutated.

Re-anchoring. Not applicable (a view mint references no tombstonable anchor; there is no `DeleteView` in this catalogue revision).

3.22 CreateMeasure

Ratified with genesis tranche G3b (`spec/CONTRACT_GENESIS_G3B_MEASURE.md`), the final rung of the genesis operation tranche (`spec/PLAN_GENESIS_OPS.md`): a staff instance's measure sequence (`StaffInstance.measures`) joins the operation-authored surfaces. Unlike Section 3.18 through Section 3.21, `Measure` is a **nested container child** of `StaffInstance`, not a `Score`-root vector — so `CreateMeasure` rides `CreateStaffInstance`'s parent-carrying shape (Section 3.12), not the four G3a families' bare-value shape.

Payload schema. `CreateMeasureOp { instance: StaffInstanceId, measure: Measure }` — the owning staff instance's identifier beside the full `Measure` value. `Measure` carries no back-pointer to its parent, so the parent must ride along, exactly like `CreateStaffInstanceOp::region`.

Canonical encoding. `instance` (canonical bytes), then the length-framed canonical bytes of `measure`. `Measure` has never been versioned — its only non-scalar field, `TimeAnchor`, has a hand-written codec with no version branching of its own — so this operation is schema major 0 unconditionally.

Reduction rule. Set-union creation, on the same discipline as `CreateStaffInstance`: a create mints the measure live, appended to the **end** of `instance.measures` (`Measure` order is append-only), if its id is fresh; a repeat create carrying a byte-identical value under the **same** owning instance reduces idempotently (`NoOpReason::AlreadyApplied`); a create whose id is already live with a differing value, **or the same measure value under a different owning instance**, is a precondition no-op with `RecreateContentMismatch` — the owning instance is part of identity here, since a `Measure` carries no parent reference of its own; a create naming a tombstoned id is a precondition no-op with `TargetTombstoned`.

Referential preconditions: the parent `StaffInstance` must be live (ungated — checked graph-aware and base-free alike, mirroring `CreateStaffInstance`'s own region check: a mint into a non-existent parent has nowhere to go even base-free); and, in both modes as Requirement 2.2 reads a referent (graph-aware only before version 0.19.0), a resolving `measure.time_signature` must name a live `TimeSignature`, and every non-`WallClock` referent of `measure.start` (an `Event`, another `Measure`, or a `Region`) must resolve. A measure stands in musical time: one whose start is anchored in musical time into a region admitting no musical offset is refused with `WrongRegionTimeModel`, as a time signature is (since version 0.19.0, before which it applied after a concurrent migration out of musical time).

Ordering and boundary-distance preconditions (core specification Graph Invariants, invariant 20). Graph-aware, against the current last live measure of the same instance (vacuous for an instance's first measure, for lack of a predecessor to compare against): the carried `start` must be **comparable** to the predecessor's start and **strictly after** it (else `MeasureOutOfOrder`), and, when a governing time signature is determinable, the **musical delta** between the two starts must equal that signature's `measure_duration()` (else `MeasureMeterMismatch`). The one exception is a pickup: while the predecessor is the instance's only live measure, it may be shorter than its bar, so the delta may also be any positive duration less than `measure_duration()` (since version 0.16.0; before it, a pickup's successor refused). A short measure after the first is not modelled: its successor still refuses. Either comparison being **unverifiable** — the two starts are not comparable, or the delta is not computable — refuses with `MeasureOrderUnverifiable` rather than silently permitting an unverifiable mint. A resolving `time_signature` that **disagrees** with the effective grid's active signature at the measure's start also refuses with `MeasureMeterMismatch`

— this agreement check is **not** predecessor-dependent and applies to a first measure like any other; `time_signature: None` skips only this agreement check, not the boundary-distance check above.

The effective grid is `StaffInstance.local_metric_grid` when present, else the enclosing region's default metric grid, reconstructed from the same operational write chains (`SetMetricGrid`'s whole-grid writes layered under `SetTimeSignature`'s per-key changes, in the order the reduction applies the writes, folding in any prospective undo restoration) in both graph-aware and base-free reduction, so the two modes never disagree. Since version 0.19.0 the order is the walk's, in which a transaction's members apply together where its first member falls; before it, the writes' stamps ordered them, so a transaction's later-stamped grid write was taken for more recent than another author's concurrent time signature that the graph applied after it, and a measure was placed by a grid the graph did not hold.

Conflict cases. None at reduction time (set-union; the differing-value re-create is a precondition gate, not a conflict).

Undo semantics. Undo of a create tombstones the minted measure and removes it from the owning instance's `measures` vector (Section 3.34). `StrictInverse` conflicts if a live reference would be stranded, across **seven** inbound surface classes: a spanner endpoint, a repeat structure's anchor sites, another measure's `start`, a meter change, a system-break advisory, a page-break advisory, or a tempo segment's `start` or optional `end`; and, since version 0.19.0, by a live later measure of the same instance, which removing any measure but the last would leave two bars from its predecessor (owned by that measure, an immutable value map). Six of the seven surfaces are owned (the referencing object must be live and not itself part of the transaction being undone); a score-level (`region: None`) tempo segment has no owning object and blocks on the reference itself. Five of the seven are restoration-aware — a prospective undo restoration that would *reinstate* a reference to the measure still blocks, and one that would *remove* a reference does not; the remaining two (another measure's `start`, and a repeat structure's anchors) are immutable value maps with no modify operation, so restoration-awareness does not apply to them. Also closes a pre-existing hole: a minted `TimeSignature` still named by a live `Measure.time_signature` now blocks its own undo, exactly as a live meter-change reference already did.

Re-anchoring. Not applicable (a measure mint references no tombstonable anchor of its own; there is no `DeleteMeasure` in this catalogue revision).

3.23 CreateTuplet

Ratified with X3.1: a `Tuplet` (core specification Tuplets as Grouping Objects) joins the operation-authored surfaces. Tuplets live in the cross-cutting registry (`CrossCuttingStructures.tuplets`) but, as repeats are not, they are *not* `CrossCuttingValue` kinds on the wire — the cross-cutting operations admit only `Tie/Slur/Beam/Spanner`, and since version 0.21.0 `Marker` and `Lyric` (Section 3.8) — so the tuplet's mint is a first-class primitive. It takes the root-level mints' bare-value shape (Section 3.21): a tuplet carries its members by identifier and has no parent container.

Payload schema. `CreateTupletOp { tuplet: Tuplet }` — the full value: identity, the `actual:notated` ratio, the member events, the optional parent tuplet, the required total sounding duration of

the members, and how its number and bracket are shown (`display`, core specification requirement `req:time:tuplet-display`).

Canonical encoding. The length-framed canonical bytes of `tuplet.Tuplet` appends `display`, a mandatory field, at schema major 4 (Binary Format companion Schema Major 4), so this operation is schema major 4 unconditionally. A `CreateTuplet` written before major 4 carries the five-field `Tuplet` of majors 0 to 3; a decoder recognizes that layout and refuses it by name rather than migrate it, since before the product's 1.0 release nothing is migrated. A degenerate ratio does not decode (core specification requirement `req:time:tuplet-ratio-construction` holds at decode as at construction).

Reduction rule. Set-union creation: a create mints the tuplet live if its id is fresh; a repeat create of a live id reduces idempotently (`NoOpReason::AlreadyApplied`), without value comparison, the cross-cutting discipline; a create naming a tombstoned id is a precondition no-op with `TargetTombstoned`. Graph-aware reduction pushes the value into the registry.

Preconditions, checked graph-aware and base-free alike from the graph-independent indices, so the two modes never disagree: the tuplet has at least one member, every member is a live event and a carried parent is a live tuplet (else `TargetMissing`); no member is a grace note, which occupies no time (else `EventDurationInvalid`, since version 0.21.0); and the members' sounding durations sum to exactly `required_total` (else `EventDurationInvalid`), the core specification's tuplet duration consistency (`req:time:tuplet-duration-consistency`), which a create may not break.

Keeping consistency. The members enter the referent index, in both modes, so the tuplet is seen wherever its members are: a `DeleteEvent` of a member must declare its tuplet compensation (re-anchoring table row "Tuplet / Member event"), its membership read from the index; a `ReplaceWithRest` compensation puts the rest in the deleted member's place in the index, as it does in the graph; and a `ModifyEvent` that would change a member's duration is refused as a precondition no-op with `EventDurationInvalid`.

Conflict cases. None (set-union; the preconditions are gates, not conflicts).

Undo semantics. Undo of a create tombstones the minted tuplet and removes it from the registry, with any decomposition attachment naming it (its reference would otherwise dangle). A member tombstoned by an undo of the transaction that inserted it declares no compensation; the tuplet it leaves can no longer fill its total, so it cascade-deletes, in the index and the graph alike, as `CascadeDeleteTuplets` would have. An undo whose restoration of an event's value would change the duration of a live tuplet's member, the tuplet not among the undo's own mints, is superseded by the tuplet's create, as a later write supersedes it: `StrictInverse` conflicts and `BestEffort` leaves the duration (since version 0.19.0, before which the restoration broke the tuplet's sum).

Re-anchoring. The tuplet references its members; the rule table's "Tuplet / Member event" row governs a member's delete. Nesting is representable (`parent`) but no producer in this revision writes it.

3.24 SetClef and SetKeySignature

Ratified with X3.6: a staff instance's clef and key sequences (`StaffInstance.clef_sequence`, `key_sequence`), which only `CreateStaffInstance` could write, become editable change by change, which authoring needs. The two kinds share one rule; this section states it for `SetClef` and `SetKeySignature` reads the same with the key for the clef.

Payload schema. `SetClefOp { instance: StaffInstanceId, offset: RationalTime, clef: Option<Clef> }` and `SetKeySignatureOp { instance: StaffInstanceId, offset: RationalTime, key: Option<KeySignature> }`: the change `offset` whole notes after the start of the instance's region, `None` to remove it. The change is placed by its offset rather than by an arbitrary `TimeAnchor`, so it names no event or measure an undo could strand; the graph holds it anchored at the region's start with that musical offset, the form the importer writes.

Canonical encoding. The instance's id (16 bytes), the length-framed canonical bytes of `offset`, then an option tag (`0` absent, `1` present) and, when present, the length-framed canonical bytes of the value. Neither value has been versioned standalone, so both operations are schema major 0 unconditionally; `KeySignature` re-validates its fifths at decode.

Reduction rule. A structural last-writer-wins register keyed by `(instance, offset)`, on the meter change's discipline (Section 3.29): a write sets, replaces or removes the one change at that position in the sequence, which stays ordered by position; a change already there, from the base or the instance's creation, is the key's predecessor. The instance must be live: missing, the operation is a precondition no-op with `TargetMissing`; tombstoned, with `TargetTombstoned`. The check reads the graph-independent object index, so both reduction modes agree. A change is written at a musical offset, so the instance's region must admit one: a region out of musical time refuses it with `WrongRegionTimeModel`, read from the coordinate discipline both modes record (since version 0.19.0). For the same reason a `CreateStaffInstance` carrying a change anchored by a musical offset into a region admitting none is refused with `WrongRegionTimeModel`, and a live instance holding a change strands a `ChangeRegionTimeModel` of its region to such a model (Section 3.11).

Conflict cases. A write concurrent with the key's last write and carrying a different value records a `StructuralFieldCollision` on `clef_sequence` (`key_sequence`), the later write in canonical order winning; carrying the same value, it is a no-op (`AlreadyApplied`).

Undo semantics. Value-restoring: undo writes back the key's predecessor — the earlier change, or none — unless a later write superseded the undone one.

Re-anchoring. None: the change names only its staff instance, whose deletion takes the sequence with it.

3.25 SetVoiceHome

Ratified with X4b (roadmap D58, Q11): a voice that visits another staff, as a cross-staff passage does, names its home staff, the one it belongs to whatever staff its notes are drawn on (core specification, schema major 5).

Payload schema. `SetVoiceHomeOp { voice: VoiceId, home: Option<StaffId> }`: the voice's home staff, `None` to clear it.

Canonical encoding. The voice's id (16 bytes), then an option tag (`0` absent, `1` present) and, when present, the staff's id (16 bytes). It embeds no versioned value, so it is schema major 0, at epoch 15.

Reduction rule. A structural last-writer-wins register keyed by the voice, seeded with no home at the voice's create and from a base's `voice_homes`. The voice must be live: missing, the operation is a precondition no-op with `TargetMissing`; tombstoned, with `TargetTombstoned`. A staff it names is a referent, checked in both modes as Requirement 2.2 reads one, a dead one refusing it with `TargetMissing`. A home names the staff a voice visits from, so one naming the staff the voice's own staff instance manifests is refused with `HomeIsOwnStaff`, read from the indices both modes keep (since version 0.21.0, after its first review; `None` is the voice at home). Graph-aware reduction writes `Score.voice_homes`; a voice's delete takes its home with it.

Conflict cases. A write concurrent with the voice's last write and carrying a different value records a `StructuralFieldCollision` on `home_staff`, the later write in canonical order winning; carrying the same value, it is a no-op (`AlreadyApplied`).

Undo semantics. Value-restoring: undo writes back the voice's earlier home, or none, unless a later write superseded the undone one or the earlier home's staff was removed since. An undo cannot remove a staff a live voice's home names, read as the undo leaves the home: `StrictInverse` conflicts and `BestEffort` keeps the staff, as for a part or a spanner naming it.

Re-anchoring. None: the home names a staff, which no operation but an undo removes, and the guard above holds it.

3.26 Repeat Structures

Ratified with the schema-major-2 revision: the dedicated authoring pair for `RepeatStructure` (core specification Chapter 5 Repeat Structures). Repeats live in the cross-cutting registry but are *not* `CrossCuttingValue` kinds on the wire — the cross-cutting operations admit only `Tie/Slur/Beam/Spanner`, and since version 0.21.0 `Marker` and `Lyric` (Section 3.8) — so the pair is a first-class primitive mirroring the cross-cutting disciplines.

Payload schema. `CreateRepeatStructureOp { repeat: RepeatStructure }` — the full value (schema major 2): identity, the `start/end` anchors, the `RepeatKind`, and the volta list. `DeleteRepeatStructureO { repeat: RepeatStructureId }`.

Canonical encoding. Create: the length-framed canonical bytes of `repeat` (the schema-major-2 layout). Delete: the bare identifier. The create's payload embeds the v2 `RepeatStructure` layout unconditionally (`kind` and `voltas` are not optional fields), so under minimal stamping a block carrying one stamps schema major 2 — the create is *born at v2*. The delete's payload is an identifier — a major-0 layout — so minimal stamping gives its blocks major 0; the kind discriminant itself is an append-only schema-*minor* vocabulary event (the same append mechanism as the Phase-3 tranche; the stamp itself always follows minimal stamping over the payload, per the Binary Format companion Schema Major 2).

Reduction rule. Set-union creation and a delete-wins tombstone, mirroring the cross-cutting family. A create mints the repeat live if its id is fresh and *every* event-referencing anchor site resolves to a live event — `start/end`, the kind's jump targets (`DaCapo.end_target`, `DalSegno.segno/end_target`) and each volta's `start/end` — since the mint must leave the graph satisfying the reference-

resolution invariants; a dead anchor is a precondition no-op (`TargetMissing`). A repeat create of a live id reads `AlreadyApplied` without value comparison (the cross-cutting discipline; the `RecreateContentMismatch` scope of Section 3.13 is unchanged). A delete tombstones the repeat (idempotent on re-delete, delete-wins); deleting a missing id is a no-op precondition failure. Graph-aware reduction adds or removes the structure from the cross-cutting registry.

Conflict cases. None at reduction time (set-union creation; delete-wins is idempotent).

Undo semantics. Undo of a create tombstones the minted repeat (Section 3.34). A delete mints nothing, so the prototype's minted-object undo does not re-introduce the tombstoned structure (P11-C8).

Re-anchoring. The structure participates in the re-anchoring rule table when a referenced event is later tombstoned: re-anchor to the nearest surviving anchor, cascade-delete only when none survives — as for spanners, spanning *every* anchor site (`start`/`end`, jump targets, volta spans). Among multiple surviving candidates — a case slurs and spanners never present, since their sole other endpoint is the forced survivor — "nearest" is currently realized as the deterministic identifier-order minimum (the same tie-break the two-endpoint collapse already used); proximity-aware (four-key) selection over a repeat's surviving sites is a deferred refinement, exactly as the spanner row defers per-kind proximity bounds. The rule row is the core specification's (Chapter 6 re-anchoring rule table, ratified with this pair).

Authoring advisory. The volta well-formedness constraints of core Chapter 5 (endings non-empty, 1-based, *strictly* ascending) are *advisory*: surfaced at authoring time under interactive validation, never enforced under reduction.

3.27 SetUserSystemBreak

Payload schema. `SetUserSystemBreakOp { region: RegionId, anchor: TimeAnchor, present: bool }` — the full anchor value (v1).

Canonical encoding. `region`, the length-framed `anchor` value, then the boolean.

Reduction rule. A last-writer-wins advisory. The break preference is recorded for the region keyed by the anchor's *resolved musical position*; graph-aware reduction adds or removes the anchor from the region's user system-break list.

Conflict cases. None (LWW advisory).

Undo semantics. Undo restores the prior advisory value for the `(region, resolved-position)` key.

Re-anchoring. Not applicable in the prototype (the advisory is keyed by resolved position; a tombstoned anchor target degrades to the region origin).

3.28 Score Settings

Payload schema. Three score-level field overwrites: `SetMetadataOp { metadata: ScoreMetadata }` overwrites the score singleton; `SetMetricGridOp { region: RegionId, grid: Option<MetricGrid> }` overwrites (or clears) a region's default metric grid; `SetUserPageBreakOp { region: RegionId, anchor: TimeAnchor, present: bool }` is the page-break sibling of SetUserSystemBreak.

Canonical encoding. Metadata: the length-framed `metadata` value. Metric grid: `region`, then an `Option` discriminant and (when present) the length-framed `grid` value. Page break: `region`, the length-framed `anchor` value, then the boolean.

Reduction rule. Three field overwrites differing only in discipline. *SetMetadata* is an **advisory** last-writer-wins: the latest write in canonical order silently wins and the operation always applies — no working state and no conflict (the same discipline as SetUserSystemBreak, on the score singleton). *SetMetricGrid* is a **structural** field overwrite keyed by region: precondition the region is live and staff-based (a FreeGraphic region has no metric-grid slot — the op is a no-op there), and reject a grid whose meter sequence names a time signature that is not live (the Chapter 5 invariant forbids installing such a grid); a grid set in a region whose time model admits no musical offset is refused with `WrongRegionTimeModel` in both modes (since version 0.19.0; clearing it still applies). *SetUserPageBreak* is a canonical LWW advisory keyed by the anchor's resolved musical position, with the same staff-based precondition. Graph-aware reduction overwrites the metadata singleton, sets the region's default metric grid, or adds/removes the page-break anchor under its resolved-position key (so two anchors resolving to one position occupy a single slot).

Conflict cases. SetMetadata and SetUserPageBreak: none (advisory LWW). SetMetricGrid: two concurrent differing grids for one region produce a `StructuralFieldCollision` on the field `metric_grid`.

Undo semantics. All three are value-restoring field overwrites (Section 3.34): undo restores the prior metadata, grid, or break preference from the key's write chain unless superseded.

Re-anchoring. The advisory breaks degrade as for SetUserSystemBreak; the metric grid and metadata are keyed by region / singleton and do not re-anchor (a deleted region's settings are no-ops — `TargetMissing`).

3.29 Meter and Tempo Overwrites

Payload schema. The finer-grained metric-model overwrites beneath the whole-grid `SetMetricGrid` (Section 3.28): `SetTimeSignatureOp { region: RegionId, anchor: TimeAnchor, time_signature: Option<TimeSignature> }` sets, replaces, or (`None`) removes the single `MeterChange` at the anchor's resolved musical position in the region's default metric grid, carrying the full `TimeSignature` value (v1); `SetTempoSegmentOp { region: Option<RegionId>, start: TimeAnchor, segment: Option<TempoSegment> }` sets, replaces, or removes the single tempo segment starting at the resolved position, in the score-level tempo map (`region: None`) or the region's local map (`Some`; a set on a region with no local map creates one).

Canonical encoding. Time signature: `region`, the length-framed `anchor`, then an `Option` discriminant and (when present) the length-framed `time_signature` value. Tempo segment: an `Option` discriminant and (when present) `region`, then the length-framed `start`, then an `Option` discriminant and (when present) the length-framed `segment`.

Reduction rule. Last-writer-wins structural overwrites keyed by `(region, resolved position)` (time signature) and `(scope, resolved start)` (tempo segment). A carried `TimeSignature` is minted set-union under the same discipline as `CreateStaff`: fresh id mints; byte-identical re-carry is idempotent; a differing value under a live id is a precondition no-op (`RecreateContentMismatch`, Section 3.13). The time-signature value's beat-group sum is validated at construction and

again at decode, so a malformed value never reaches reduction. A tempo-segment write preconditions that the *resulting* map is well-formed (segments ordered and non-overlapping; a non-constant shape carries its end data; the carried segment's own start equals the operation's `start` key) — a write that would malform the map is refused as a precondition no-op (`TempoMapMalformed`). A time signature written in a region whose time model admits no musical offset is refused with `WrongRegionTimeModel` in both modes (since version 0.19.0, before which it applied and left a meter in musical time). Graph-aware reduction applies the meter change to `default_metric_grid.meter_sequence` and the segment to the scoped tempo map.

A tempo segment whose start or end anchor names a region that is not live is refused with `TargetMissing`, the region read as a referent in both modes (Requirement 2.2), and one anchored by a musical offset to a region admitting none with `WrongRegionTimeModel` (both since version 0.19.0, before which a score-level segment's anchors were not read).

Conflict cases. Two concurrent differing writes of one key produce a `StructuralFieldCollision` on the field `meter_sequence` or `tempo_segments` respectively, with the standard winner/loser recording; identical concurrent writes reduce idempotently.

Undo semantics. Value-restoring per Section 3.34: undo restores the key's chain-predecessor value (or its absence).

Re-anchoring. An event-anchored `anchor/start` whose event is later tombstoned degrades by the framework's anchor rules; the overwrite keys on the *resolved* position, so the recorded change survives its anchor.

Open Question

P12-C5. A mid-region meter change authored by `SetTimeSignature` reduces cleanly and materialises into the grid, but the notational-decomposition pre-pass currently honours only a region's *first* governing meter (P12-H4's single-meter simplification), so the derived notation ignores the change until multi-meter decomposition lands. The reduction-level semantics are pinned here; the derived-annotation gap is P12-H4's.

3.30 SetStaffLayout

Payload schema. `SetStaffLayoutOp { staff_instance: StaffInstanceId, instrument_override: Option<InstrumentId>, staff_lines_override: Option<StaffLineConfiguration>, visible: bool }` — the non-break layout advisories with a graph home: the staff instance's three inline advisory fields, overwritten as a unit.

Canonical encoding. `staff_instance`, an `Option` discriminant and (when present) `instrument_override`, an `Option` discriminant and (when present) the length-framed `staff_lines_override`, then the boolean.

Reduction rule. A last-writer-wins *advisory* overwrite keyed by `staff_instance`. Preconditions: the staff instance is live; a present `instrument_override` resolves to a live instrument under graph-aware reduction. The richer engraving-override vocabulary (stem direction, notehead shape, custom positions) has no durable graph home yet and remains projected layout

state — extending this primitive to cover it is staged with the data-model expansion.

Conflict cases. None (LWW advisory).

Undo semantics. Value-restoring per Section 3.34.

Re-anchoring. If the staff instance is tombstoned, the overwrite is a no-op (`TargetTombstoned`).

3.31 DeclareTransaction

Payload schema. `TransactionDescriptor { id: TransactionId, label: String, category: Option<TransactionCategory> }`. Value-complete in v0 and unchanged.

Reduction rule. Records the descriptor. Member primitives reference the transaction id and **MUST** causally depend on the descriptor; the members reduce atomically (all-or-nothing) in canonical order.

Conflict cases. A missing descriptor or a member that does not causally follow it produces a `TransactionConflict`; any member failure rolls back the whole transaction and all members read `NoOp{TransactionConflict}`.

Undo / re-anchoring. Transactions are the unit of undo (Section 3.34); re-anchoring is per member.

3.32 ResolveConflict (meta-operation)

Payload schema. `ResolveConflictPayload { target: ConflictId, action: ResolutionAction }`. Value-complete.

Reduction rule. Transitions the target conflict's resolution state. An action of `Dismiss` reaches the `Dismissed` state; any other action reaches `Resolved`. Re-resolving with the same action is idempotent; two concurrent resolves with differing actions produce a meta-conflict.

Beyond the concurrent case (ratified Pass 12, closing P12-K4). The earliest-applied-resolve-governs rule is *universal*: a causally-later resolve with a differing action does not supersede the first — it reduces `AlreadyApplied` — and *any* resolve targeting a `Dismissed` conflict likewise reads `AlreadyApplied`. Intentional re-resolution is deliberately outside the v1 operation set; a future dedicated operation (a `ReopenConflict`-class primitive) is the sanctioned path if it is ever needed. The meta-conflict record names both resolver operation ids in `caused_by`; conflict records themselves have no `TypedObjectId` kind (deliberate — they are materialized state, not graph objects), so the contested conflict is identified by the `equivocation`-style field key, not an object reference.

Rationale

Pass 11 added `ResolutionAction::Dismiss` (item 2.5) precisely so the `Dismissed` state is reachable by an authored operation rather than merely representable. The catalog records that `Dismiss` is the action that selects it (resolving the v0 ambiguity P11-C10).

3.33 ResolveEquivocation (meta-operation)

Payload schema. `ResolveEquivocationPayload { target: OperationId, chosen: EnvelopeHash }` — the equivocated slot and the candidate envelope (by canonical-bytes hash) that shall stand. Value-complete.

Canonical encoding. `target` (16 canonical bytes), then `chosen` (32 bytes), per the codec baseline.

Reduction rule. Order-independent promotion of an equivocated slot (core specification Chapter 6, Equivocation): when the operation set holds an `Equivocated` slot for `target` and `chosen` names one of its candidates, the slot reduces as if it had always been `Single` with the chosen envelope — the chosen candidate contributes to canonical reduction at its own canonical position, and operations that were pending on the equivocated id unblock. Among multiple resolves naming the same slot, the one earliest in canonical order governs; a later resolve naming the *same* candidate reduces idempotently (`AlreadyApplied`). The resolve operation must itself occupy a `Single` slot; an equivocated resolve is excluded from reduction like any other equivocated slot. A resolved slot records no `OperationSlotEquivocated` anomaly; the losing candidates remain in the diagnostic candidate store only.

Conflict cases. Two resolves of one slot naming *differing* candidates produce a `StructuralFieldCollision` meta-conflict on the field `equivocation_resolution`, recording the governing resolve (earlier in canonical order) as winner and the later as loser, with both operations in `caused_by` — the same discipline as `ResolveConflict` meta-conflicts. Preconditions: a resolve whose `target` is not an equivocated slot, or whose `chosen` is not among the slot's candidates, is a precondition no-op.

Edge semantics (ratified Pass 12, closing P12-K6). Promotion is *single-pass*, not fixpoint: a promoted candidate that is itself a `ResolveEquivocation` does not govern a further promotion in the same reduction. A resolve in a *quarantined* replica segment is excluded from reduction and never governs. A resolve held *pending* by its own causal gaps still governs promotion — the verdict is a pure function of the slot map (set-level), while the resolve's own effect stays pending. The invalid-target/invalid-chosen no-op reuses `TargetMissing`; a dedicated reason was considered and rejected (the appended-reason budget is spent where a distinct verdict changes caller behavior, which it does not here).

Undo semantics. Mints nothing; not inverted under the prototype's minted-object undo (P11-C8).

Re-anchoring. Not applicable (the payload references an operation slot, not a graph object).

Rationale

The core specification names three resolution paths for an equivocated slot: transport-level reconciliation, this explicit operation, and a profile-declared deterministic selection policy. This entry pins the schema for the explicit-operation path, which the core specification previously named only in prose. The profile-policy path is *deferred with a named landing site* (Pass 12 disposition of P12-K5): v1 profiles declare *no* selection function, and the hook's definition belongs to the Profile Conformance companion when it is written — the reducer deliberately carries no policy hook until a profile can declare one.

3.34 UndoTransaction (meta-operation)

Payload schema. `UndoTransactionPayload { target: TransactionId, policy: UndoPolicy }`, with `UndoPolicy` one of `StrictInverse`, `BestEffort`, `Cascade`. Value-complete.

Reduction rule. A forward compensating edit computed against the materialised state at the undo's canonical position (never literal time travel). A transaction is undone once it is complete: an undo that is a member of the transaction it names is refused with `TargetMissing`, read from the envelope, and so fails the transaction holding it (since version 0.19.0, before which it reversed the members before it, an earlier undo's restorations among them, against objects that undo had removed). The compensation has two parts.

Minted-object tombstoning (as before): every object the target transaction minted is tombstoned. `StrictInverse` conflicts (`TombstonedTarget`) if any minted object was already tombstoned; `BestEffort` tombstones the survivors.

Value restoration (this revision): for every last-writer-wins overwrite the target transaction performed — event and identified-pitch modification, respelling, cross-cutting modification, metadata, canvas layout defaults, spelling precedence, tuning context settings, metric grid, meter change, tempo segment, staff layout, and the user break advisories — the reducer maintains, per overwritten key, the *canonical-order write chain* of (writer, value) pairs. Undoing the transaction restores each written key to its chain-predecessor value (or its absence, where the transaction introduced the first value), *provided the transaction's write is still the key's last writer*. Because the chain is keyed by canonical order, the restored value is a pure function of the operation set: permutation-invariant by construction. When a causally-later or canonically-later write has superseded the key, `StrictInverse` refuses the whole undo with a `TransactionConflict` conflict naming the undo and the superseding writer; a cross-cutting value whose restoration would name an object tombstoned since (an endpoint event, a spanner's measure or region anchor, a staff it spans) is superseded by that tombstoning (since version 0.19.0, before which the dangling value was restored); a value whose restoration would write in musical time into a region a migration has since taken out of it (a break anchored by a musical offset, a meter change, a metric grid, a tempo segment anchored in the region, a clef or key change of its staff instance) is superseded by that migration, as its setter is refused there (since version 0.19.0, before which it was restored at an offset the region no longer admits); and a value whose restoration would name an object tombstoned since (a meter change's or metric grid's time signature or anchor, a tempo segment's anchors, an instrument override, a break's anchor) is superseded by the operation that tombstoned it, so an undo of an undo restores no value naming what the first undo removed (since version 0.19.0, before which it restored the dangling reference); `BestEffort` restores the still-last-written keys and skips the superseded ones. Restorations are expressed in the effect status (a fully clean compensation is `Applied`; a mixed one is `AppliedWithRepair` carrying only the tombstone repairs) — no new repair vocabulary.

Strand-blocks (*ratified Pass 12, closing P12-K10*). A `StrictInverse` undo that refuses to tombstone a minted object still referenced by a live non-member (e.g., a staff whose instance survives outside the transaction) records `ConflictKind::TransactionConflict` — the reuse is blessed: the strand-block *is* a transaction-scoped conflict of the undo, and the conflict record's affected objects and description carry the strand detail. No dedicated undo conflict kind is added. A `BestEffort` undo keeps such an object instead, and keeps what a kept object names in turn: the strand-block is read against the objects still going until it keeps none (since ver-

sion 0.19.0, before which a staff kept for its live instance lost the instrument it names, read as going with it). It also keeps a time signature that a meter change or grid it leaves in place still names: best effort applies only the invariant-20-safe subset of its meter restorations, chosen once its mints are gone, so a restoration it drops leaves its key's value, and a signature that value names stays, the undo taken again without it (since version 0.19.0, before which the signature was removed and the meter change named nothing). A region is held by a live marker anchored to it, and by one on an event the undo removes that the repair would orphan to the region's start, read as the repair will leave it, each read as the whole undo leaves it, its restoration included (Section 3.8; since version 0.21.0).

Idempotence asymmetry (*ratified Pass 12, closing P12-K11*). An undo's value restorations enter the write chains as ordinary writes by the undo operation — no distinguished undo provenance. Consequently a *second* undo of the same transaction finds each restored key superseded by the first undo and refuses (`Conflicted` under `StrictInverse`, skipped under `BestEffort`), while *absence* restorations (not representable as chain writes) repeat idempotently. The asymmetry is normative, documented behavior; a chain-native undo provenance would be revisited only under the deferred undo-as-operation (streaming-consistent undo) design, which subsumes this question.

Still deferred (P11-C8, narrowed): re-introducing content tombstoned by *delete* primitives (a deterministic resurrection needs a system-derived identifier derivation the ratified closed tag set does not yet include); and `Cascade`'s dependent-closure computation — `Cascade` remains `StrictInverse` over the same set. `Transpose` inversion is no longer listed: `TransposeInterval` is undone by value restoration, not by applying an inverse interval (Push 4a, closing P12-K2). An inverse $(-d, -c)$ does exist wherever both components are negatable — the reduction never saturates — but `i32::MIN` has none, so the inverse is partial and undo does not depend on it. The frozen `Transpose` is undone by neither.

Chapter 4

v0 → v1 Payload Migration

A v0 envelope carries an identifier-only payload; a v1 envelope carries the value-typed payload this catalog defines. The two forms do not coexist as permanent dialects (that would double the reducer surface forever); instead the catalog ships a **one-time migration** that lifts a v0 envelope to v1 using the score graph as context, applied once on read. Production code carries only v1 payloads; v0 envelopes survive only as a regression corpus.

Requirement 4.1

The migration `migrate_v0_envelope(v0, context: &Score)` **MUST** be **deterministic** (two implementations migrating the same v0 envelope against the same context produce byte-identical v1 envelopes) and **equivalence-preserving** (a v0 envelope and its v1 migration reduce to byte-identical canonical `MaterializedState`). When a value cannot be reconstructed from the v0 projection plus the context, the migration **MUST** report the envelope unmigratable rather than fabricate a value, and the bundle opens read-only.

The reference implementation (`epiphany-ops::migrate`) reconstructs the `InsertEvent` event, the `DeleteEvent` compensation, the `ChangeRegionTimeModel` model, the `SetUserSystemBreak` anchor, and the event-anchored cross-cutting structures (`Tie` / `Slur` / `Beam`) self-containedly from the v0 projection; a `Spanner`, anchored by `TimeAnchor`s rather than event endpoints, remains unmigratable until the projection carries them. It recovers a `RespellPitch` spelling from the context (P12-K1, Section 3.3). The migration's merge gate (`epiphany-testkit::migration`) drives the inverse direction — projecting a v1 corpus to v0 and migrating it back — and asserts byte-identical reduction plus a non-vacuity guard.

Chapter 5

K1 — Framework Slots (Phase 3)

This chapter drafted the remaining catalogue items as framework slots. The Phase-2 **M2** expansion (the broad-K0 groups in `epiphany-ops`) implemented four groups of them; with the **M2e** catalogue expansion their full per-primitive schemas now appear in Chapter 3, so they are *normative under the Phase-2 profile* and an implementation **MUST** *not* reject them. They are cross-referenced first. The genuinely Phase-3 slots that remain **unavailable** are listed second: an implementation **MUST** reject an operation of one of *those* kinds. Each remaining slot is a schema-fill against the template of Chapter 2; adding one is not a fresh design.

Implemented since M2 (now in Chapter 3)

Modify event; identified-pitch operations; transpose M2 Group 1 — Sections 3.4, 3.5, and 3.6.

Delete / modify cross-cutting M2 Group 2 — Sections 3.9 and 3.10 (creation is Section 3.8).

Create / delete region / staff instance / voice M2 Group 3 — Section 3.12 (set-union creation and the empty-only delete).

Set metadata / metric grid / user page break M2 Group 4 — Section 3.28 (advisory metadata, structural metric grid, advisory page break).

Implemented in the Phase-3 first tranche (now in Chapter 3)

Create staff; set time signature / tempo segment; set layout Sections 3.13, 3.29, and 3.30. The disciplines are as drafted: set-union creation, LWW structural overwrite, and LWW advisory respectively.

Added in the schema-major-2 revision (Chapter 3)

Create / delete repeat structure Section 3.26 — net-new primitives (never drafted as framework slots). Set-union creation with the all-anchors-live precondition and a delete-wins tombstone; the create is born at v2, the delete stamps major 0.

Retired slots (ratified Pass 12: outside the operation set)

Create score / canvas — retired, closing P12-K8; narrowed by `spec/RULING_GENESIS_PERSISTENCE.md`

The document root and the canvas remain *structural givens*, not operation products: `TypedObjectId` has no Canvas kind, and the root is never op-minted (core specification, Chapter 5 The Canvas). `RULING_GENESIS_PERSISTENCE` reverses only the wider claim this slot's description used to carry — that the score's *contents* arrive outside the operation set. They do not: the genesis tranche makes every mutable `Score` field operation-authored, `Canvas.layout_defaults` among them (Section 3.15). What survives, and is load-bearing, is the narrower claim above — the canvas *itself* is still never minted, addressed, or deleted by any operation, and no `TypedObjectId` kind exists for it. These are not "unavailable slots" awaiting a design — no operation kind will be assigned to them. The decision is revisited only if an addressable multi-canvas model is adopted at a future schema major.

Non-Goal

The full 60–80-primitive catalogue is not a Phase-2 deliverable. The framework (Chapter 2) and the K0 representative set (Chapter 3) are sufficient to exercise every reduction discipline; the remaining primitives are Phase-3 schema-fill.